arXiv:2609.21527v1 [cs.LG] 18 Sep 2026

OPENMAS-GCOM. A DIAGNOSTIC BENCHMARK FOR GRAPH-ENHANCED MULTI-AGENT SYSTEMS

Kairui Yang*, **Xunkai Li***, **Kaixiang Zhang**,
Minghao An, Zekai Chen, Yuxuan Ba, Rong-Hua Li
*Equal contribution.

ABSTRACT

Graph-enhanced multi-agent systems (G-MAS) coordinate large language model agents through communication graphs and role assignments, which determine how agents exchange information and divide responsibilities. However, final-score comparisons across systems combine differences in models, communication patterns, roles, and computation costs, making performance differences difficult to attribute to specific communication structures, role assignments, and information flows. To address this evaluation attribution problem, we introduce OpenMAS-GCom, a benchmark for diagnosing how these components affect G-MAS performance through controlled interventions. We represent systems through collaboration units, communication links, shared intermediate information, and execution rules. OpenMAS-GCom compares original systems with versions modified by changing one component while keeping tasks, models, prompts, and budget limits fixed. We rewire communication edges, remove specialist or critic agents, replace intermediate messages with incorrect content, and disable workers during execution. The benchmark evaluates 17 single-agent, ordinary multi-agent, and graph-enhanced configurations on 29 datasets across six domains. We add 400 G-MAS-Complex tasks requiring agents to combine information from multiple documents, resolve conflicting records, and return specified values with source identifiers. Experiments show larger mean losses after specialist removal than after critic removal, different performance degradation under incorrect messages and worker failures despite similar original scores, and different configurations achieving the highest accuracy and accuracy per token on G-MAS-Complex.

1 INTRODUCTION

Large language model (LLM) agents collaborate by assigning roles and exchanging intermediate results to solve mathematical reasoning, code generation, and question answering tasks (Wu et al., 2023, Du et al., 2023). In graph-enhanced multi-agent systems (G-MAS), communication graphs determine which agents exchange information, while roles determine which agents generate, verify, or integrate solutions (Zhuge et al., 2024, Zhang et al., 2025). For example, one agent proposes an answer, another checks it, and a third combines intermediate results. Performance therefore depends on both the underlying language model and how agents communicate and divide responsibilities.

Existing evaluations mainly compare final task scores across systems. Recent studies analyze information propagation across communication topologies, while MultiAgentBench evaluates task completion under different multi-agent settings (Shen et al., 2025, Zhu et al., 2025). A higher score may accompany a stronger model, more agents, different communication links or roles, and greater computation costs. These factors often change together across methods, making performance differences difficult to attribute to specific organizational choices. This creates an evaluation attribution problem for G-MAS. How do communication structures, role composition, and information flow affect performance when other evaluation settings are held fixed?

To address this problem, we present OpenMAS-GCom, a diagnostic benchmark using controlled organizational interventions. We represent each system through collaboration units, communication links, shared intermediate information, and execution rules. We compare original and modified systems after changing one component, with task inputs, model settings, prompts, and computa-

tion budget limits fixed. We evaluate 17 single-agent, ordinary multi-agent, and graph-enhanced configurations on 29 datasets across six domains through a common interface that records outputs, intermediate messages, executed units, and resource use. We introduce 400 G-MAS-Complex tasks to examine how agents combine information from multiple documents, resolve conflicting records, and return specified values with source identifiers. Four intervention protocols rewire communication edges while preserving node degrees, remove specialist or critic agents, replace intermediate messages with incorrect content, and disable workers during execution.

Our Contributions. (1) **Comprehensive Benchmark.** We integrate datasets, baseline configurations, and four intervention protocols for evaluating changes to communication, roles, intermediate information, and worker availability under shared settings. (2) **Valuable Insights.** Across five datasets, specialist removal reduces the mean score by 3.58 percentage points, compared with 0.58 for critic removal. MAD and VeriMap start with similar accuracy but respond differently to incorrect messages. On G-MAS-Complex, different configurations achieve the highest accuracy and accuracy per token. (3) **Open-sourced Benchmark Library.** We organize task loading, method execution, scoring, and interventions into reusable modules. Run identifiers link configurations, predictions, intervention settings, and costs to support reproducible evaluation.

2 Problem Statement

OpenMAS-GCom connects task evaluation with organizational diagnosis through two complementary pipelines. The collaboration pipeline produces predictions and execution records through a common interface. The diagnostic pipeline measures how these outcomes change when selected organizational components are modified.

2.1 End-to-End Collaboration Pipeline

Given a task domain T with evaluation set $D_T = \{(x_i, y_i)\}_{i=1}^{N}$, a method F specifies organization construction and execution. Under configuration λ, its construction procedure produces $\mathcal{O}_i = F_{\text{build}}(x_i, \lambda)$. A fixed design uses the same organization across inputs, while task-dependent construction can change its units and connections. Each organization has the form

$$\mathcal{O} = (V, E, S, \pi), \tag{1}$$

where V specifies runnable collaboration units, E specifies permitted directed communication, S holds shared information and intermediate states, and π governs activation, routing, state updates, and termination. A unit declares its role, local state, and input and output schemas. Agent units form a subset $V_A \subseteq V$, alongside tool or control units. These components separate participation, communication, information access, and execution decisions.

For input x_i, sequential workflows activate units in order, parallel workflows activate several eligible units, and debate repeats message exchange through the policy. Graph-based execution uses the configured relations to select permitted transfers and activations. The runner records the trace

$$\tau_i = ((v_t, m_t, a_t))_{t=1}^{T_i}, \tag{2}$$

where v_t is the executed unit, m_t is its available message state, and a_t is its output or control action. The final output o_i is extracted from the completed trace. Associated records contain token usage, model calls, latency, executed units and relations, retries, and termination status.

The representation captures the organizational properties exposed to the runner. Method-specific prompts, learned parameters, tool definitions, and controller logic remain in the configuration and policy. Shared-state permissions specify the information each unit may read or write. The configured organization describes available units and relations, while the trace identifies those used for an input. This distinction connects the comparison interface with configuration-specific execution details.

The task score is

$$s(F, \mathcal{O}_T, T) = \frac{1}{N} \sum_{i=1}^{N} M_T(P(o_i), y_i), \tag{3}$$

where P parses the output, M_T is the task-specific metric, and $\mathcal{O}_T = \{\mathcal{O}_i\}_{i=1}^{N}$ collects the task-specific organizations. The parser and metric connect execution with comparable task scores.

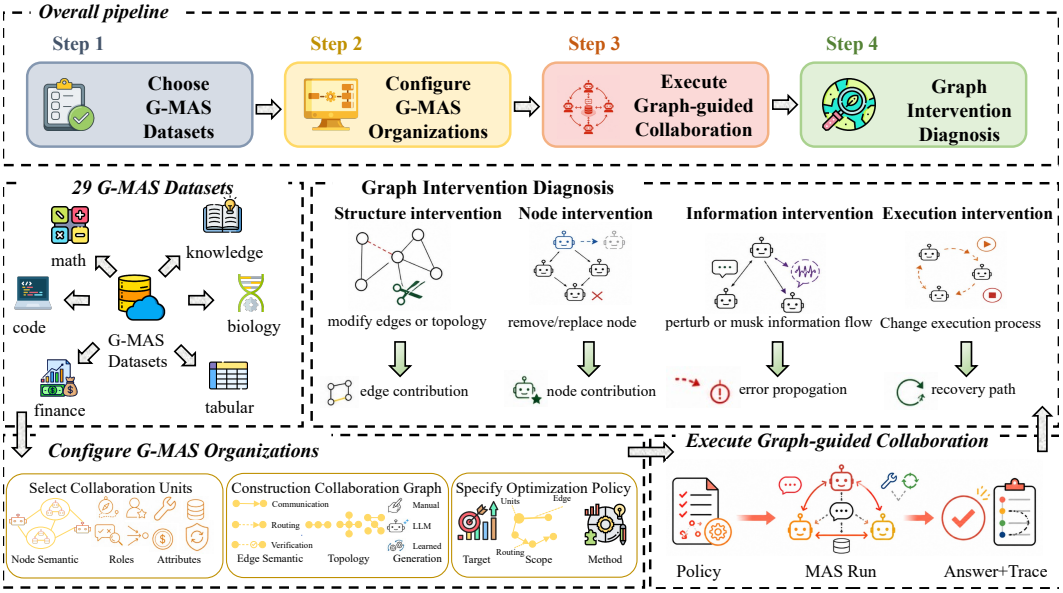


Figure 1. OpenMAS-GCom model framework. The benchmark selects datasets, configures G-MAS organizations, executes graph guided collaboration, and applies four graph intervention protocols.

2.2 Two-Stage Diagnostic Pipeline

The reference stage executes the original organization and records predictions, traces, and costs. The intervention stage modifies one organizational component and repeats the evaluation on the same tasks. Shared controls are

$$\mathcal{C} = \{D_T, \Theta, M_T, P\}, \tag{4}$$

where Θ contains the backend, prompts, decoding and routing settings, stopping rules, and task-level execution and communication budget limits retained within each pair. For intervention type r, the modified organization is

$$\mathcal{O}'_{r,i} = I_r(\mathcal{O}_i, \Delta\mathcal{O}_{r,i}). \tag{5}$$

The paired intervention response compares the resulting task scores

$$\Delta_r(F, T) = s(F, \mathcal{O}'_{r,T}, T) - s(F, \mathcal{O}_T, T). \tag{6}$$

Here $\mathcal{O}'_{r,T} = \{\mathcal{O}'_{r,i}\}_{i=1}^{N}$ collects the modified organizations. The paired score change measures the average response to intervention r under the controls in Equation 4. A negative value indicates lower performance after modification. Its unit of analysis is the complete modified execution, including subsequent message processing and actions. Structure, node, information, and execution interventions modify relations, units, intermediate messages, and worker availability, respectively.

3 Benchmark Design

OpenMAS-GCom integrates task selection, organization configuration, collaborative execution, and paired diagnosis. Figure 1 connects task inputs, execution records, and metrics.

3.1 Task and Dataset Ecosystem

Broad Task Suite. The benchmark covers 29 datasets across mathematics, knowledge, code generation, biomedical question answering, finance, and tabular reasoning. These domains combine different requirements for sequential reasoning, specialist knowledge, executable checking, and structured information integration. Dataset loaders provide task inputs, reference answers, and metric metadata to the common runner.

G-MAS-Complex Construction. G-MAS-Complex contains 400 frozen multi-document tasks in four complexity tiers. Document dependencies distribute relevant information across sources, conflicting records require revision or authority resolution, and structured outputs require consistency

checks. These properties combine retrieval, reconciliation, reasoning, and verification within each task. Comparisons measure how the evaluated configurations handle these coupled requirements. Tier labels describe task construction, and fixed-method scores measure empirical difficulty.

The four tiers contain 100 tasks each and share a structured answer contract. In an illustrative inventory task, D1 records 12 units, D2 revises the inventory to 9, and D3 reserves 4. The answer requires 5 available units and source identifiers D2 and D3. Selecting D1 gives 8 units, while omitting D2 produces an incomplete source record. Strict exact match checks required fields, values, rankings, references, checkpoints, and checksum constraints together. Appendix B.1 expands this example and distinguishes content errors from output-format failures.

3.2 Algorithm Taxonomy

Definition of G-MAS. A graph-enhanced configuration contains at least two agent units and an explicit directed graph that constrains routing, activation, verification flow, or execution order. Classification uses the graph exposed to the adapter and the runtime decision controlled by its edges. Fixed, task-generated, and learned graphs can satisfy this criterion. Equation 1 supplies the representation used to inspect these properties.

Single-Agent and Ordinary Multi-Agent Configurations. Single-agent configurations use one agent without inter-agent communication, represented here by DeepSeek and Qwen (DeepSeek-AI, 2025, Qwen Team, 2025). Ordinary multi-agent configurations coordinate agents through their dialogue or collaboration policies. The tables place the evaluated AutoGen, MADebate, and DyLAN configurations in this group (Wu et al., 2023, Du et al., 2023, Liu et al., 2024). These labels identify the reported configurations, while the adapter description specifies the organization and execution mechanism for each method.

Graph-Enhanced Configurations. The selected graph-enhanced configurations cover graph construction, sparse interaction, routing, verification, and learned coordination. Table 5 lists the corresponding methods and sources. Configuration records describe graph materialization, construction rules, runtime use, role heterogeneity, and recovery. These attributes connect method selection with the intervention protocols.

GPTSwarm optimizes prompts and graph connectivity (Zhuge et al., 2024), while G-Designer learns a generator for task-dependent communication graphs (Zhang et al., 2025). Configuration attributes further distinguish topology construction, runtime updates, learned components, role heterogeneity, verification, and recovery. Appendix B.4 defines these attributes and links graphs with execution.

3.3 Graph Intervention Protocols

Each protocol modifies one organizational factor under the shared controls of Equation 4. The four protocols examine where information travels, which units process it, what they receive, and how execution continues when workers become unavailable.

Structure Intervention. The structure protocol rewires directed edges while preserving the agent set, in-degree and out-degree sequences, edge count, and communication budget limit. A directed edge swap exchanges the destinations of two connections, changing communication partners while retaining local degrees. Self-loops, duplicate edges, and invalid role connections are rejected. The score change measures the response to this modification. Executed calls and messages describe how the resulting graph uses its available budget.

Node Intervention. The node protocol removes selected collaboration units and their incident edges. The No critic and No specialist variants target different role groups. Expert only retains the expert nodes and their induced connections. Each variant links its available functions and communication relations with its task score.

Information Intervention. The information protocol replaces eligible upstream records or intermediate candidates with format-valid incorrect candidates. The modified content reaches downstream units through the existing graph and routing rules. The corruption rate is the fraction of eligible items replaced. This protocol measures final-answer accuracy under incorrect intermediate inputs.

Execution Intervention. The execution protocol disables workers during runtime and resumes from the last valid shared state through configured retry, fallback, or rerouting procedures. Increasing the failure count tests how larger disruptions affect task completion. Appendix A.2 provides the intervention settings and shared controls.

3.4 Evaluation Protocols

Utility. We compare single-agent, ordinary multi-agent, and graph-enhanced configurations on shared tasks. Single-agent and ordinary multi-agent results provide reference scores for assessing the utility of each configuration. These system comparisons include the complete reasoning and execution procedure. Paired interventions examine changes within a configured organization.

Effectiveness. We examine task scores and method rankings across the six domains and G-MAS-Complex. Domain summaries describe performance across related tasks, while individual dataset scores capture more specific requirements. These views identify the strongest configurations and the separation among competing methods.

Robustness. We compare each modified score with the same configuration's reference score. Absolute score change measures the difference in task units, and relative retention divides the modified score by a positive reference score. Rewiring ratios, unit-removal variants, corruption rates, and worker-failure counts define the intervention settings.

Efficiency. We record token usage, model calls, latency, executed units and relations, and retries. For G-MAS-Complex, strict exact match describes successful completion, while token usage and latency describe execution requirements. All four dimensions use the common execution interface and task-specific scoring rules.

4 Experiments and Analyses

Three questions organize the evaluation. **Task utility** covers Q1 to Q3. **Organizational sensitivity** covers Q4 to Q6. **Failure robustness and resource use** covers Q7 and Q8. The following experiments examine each group through specific comparisons. **Q1 (Utility).** Which evaluated configurations improve task scores over single-agent baselines? **Q2 (Utility).** How do the best evaluated graph-enhanced configurations compare with ordinary multi-agent systems? **Q3 (Effectiveness).** How do existing G-MAS methods perform across different task domains? **Q4 (Structure Intervention).** Does topology matter when the degree sequence and communication budget limits are fixed? **Q5 (Unit Intervention).** How much do role and collaboration-unit removals affect performance? **Q6 (Information Intervention).** How does accuracy change as the fraction of incorrect upstream items increases? **Q7 (Execution Intervention).** How robust are G-MAS executions as the number of failed workers increases under recovery? **Q8 (Efficiency).** How efficient are G-MAS methods?

A. Experimental Setup

Datasets. We evaluate seven categories. (1) Mathematics uses GSM8K and AQuA (Cobbe et al., 2021, Ling et al., 2017). (2) Knowledge uses MMLU-Pro and StrategyQA (Wang et al., 2024, Geva et al., 2021). (3) Code uses HumanEval and LiveCodeBench (Chen et al., 2021a, Jain et al., 2024). (4) Biomedical QA uses MedQA and MedMCQA (Jin et al., 2021, Pal et al., 2022). (5) Finance uses ConvFinQA and FinQA (Chen et al., 2022, 2021b). (6) Tabular reasoning uses TabFact and WikiTableQuestions (Chen et al., 2020, Pasupat & Liang, 2015). (7) G-MAS-Complex contains 400 frozen multi-document tasks in four tiers. Full definitions are in Appendix A.1.

Baselines. We evaluate the 17 configurations in Table 11 through common adapters and evaluation controls. Appendix A.2 describes execution, and Appendix B.5 defines validation records.

Scores and variation. The broad suite retains dataset-specific metrics, and G-MAS-Complex uses Strict EM. Table 1 reports central scores. Appendix A.1 separates sample-set variation, run variation, recorded estimates, and binomial standard errors. Rankings in Q1 to Q3 describe displayed central values. Intervention analyses report paired score changes and reference-score retention, while efficiency compares observed scores and resource use.

B. Utility of Multi-Agent Collaboration (Q1)

To answer Q1, we compare Single-Agent, Ordinary MAS, and Graph-enhanced MAS on 13 datasets.

In Table 1, graph-enhanced configurations have the highest displayed central score on 7 of 13 datasets, single-agent configurations on 4, and ordinary multi-agent configurations on 2. Single-agent models have the highest central scores on both displayed code datasets and ConvFinQA. This

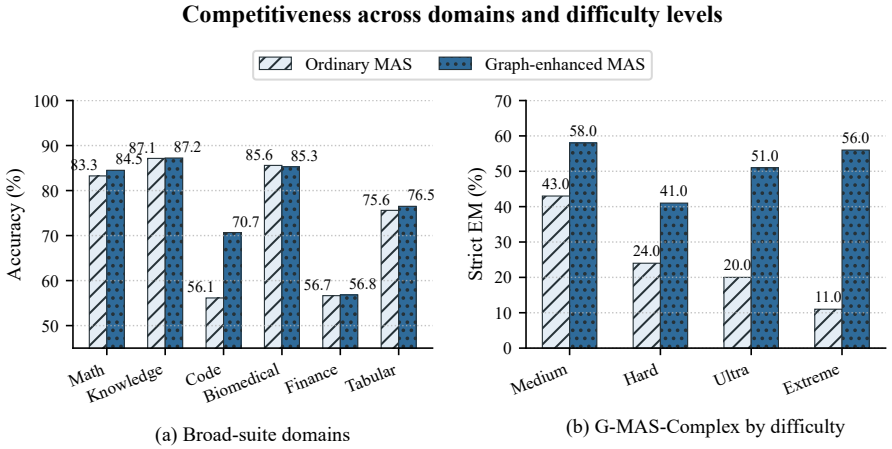
Table 1. Central-score comparison (%). Bold and underlined entries mark the highest and second-highest distinct displayed scores, including ties. Appendix A.8 defines the statistical sources.

	Mathematical Reasoning		Knowledge & Commonsense		Code Generation		Biomedical QA		Financial Reasoning		Tabular Reasoning		Complex Collaboration
Method	GSM8K	AQuA	MMLU-Pro	StrategyQA	HumanEval	LiveCode	MedQA	MedMCQA	ConvFinQA	FinQA	TabFact	WTQ	G-MAS-Complex
DeepSeek	93.03	89.89	<u>73.07</u>	84.60	83.54	21.90	88.58	**81.27**	60.13	38.10	91.47	76.73	<u>26.75</u>
Qwen	59.44	68.90	60.93	79.40	**90.24**	**26.86**	78.95	73.53	**61.81**	28.51	81.40	65.27	13.25
AutoGen	92.92	<u>90.16</u>	71.80	84.13	81.71	18.86	88.74	79.67	60.67	36.70	91.67	75.93	23.50
MADebate	93.20	89.89	71.87	84.20	79.88	20.38	89.45	79.67	61.01	36.97	91.47	76.67	14.00
DyLAN	93.33	89.89	**73.87**	**86.67**	85.98	22.86	88.45	79.93	61.10	<u>38.88</u>	<u>92.53</u>	77.40	4.50
GPTSwarm	<u>93.66</u>	**91.34**	71.20	85.67	85.57	21.14	<u>89.63</u>	<u>80.20</u>	61.12	38.62	**92.60**	76.60	15.50
GDesigner	<u>93.66</u>	89.63	71.87	85.00	85.37	<u>23.43</u>	88.85	79.00	<u>61.54</u>	38.51	91.73	**80.33**	14.50
SparseCT	**93.86**	89.76	71.33	85.80	85.98	20.76	88.92	79.67	60.81	**39.23**	91.93	77.47	15.25
R-GFM	93.63	90.03	71.53	84.40	89.02	21.52	89.47	79.33	60.63	38.71	92.13	<u>79.80</u>	2.25
BigMAS	93.45	89.63	72.20	85.53	85.77	21.33	**89.87**	78.73	60.69	38.36	92.33	76.80	19.25
GoAgent	93.23	89.76	70.33	85.27	79.88	21.14	89.40	79.13	60.49	37.20	92.33	76.20	**50.25**
ARGDes	93.48	89.63	71.47	<u>86.00</u>	<u>89.63</u>	21.33	88.77	79.20	60.20	38.71	91.93	77.00	13.25
EIB	93.33	89.37	72.47	85.40	83.54	21.33	89.00	79.00	61.34	38.54	92.33	79.33	14.00

distribution places configuration choice in the context of the target task. Our Conclusion (C1). Central-score advantages vary across tasks and configuration groups.

C. Competitiveness of Graph-Enhanced MAS (Q2)

For Q2, Figure 2 displays group-wise maxima over 3 ordinary and 12 graph-enhanced configurations. Each bar summarizes attainable reported scores in its group and task subset. The selected configuration can change across domains and tiers, so the comparison describes the best displayed score in each setting.



(a) Broad-suite domains

(b) G-MAS-Complex by difficulty

Figure 2. Group-wise maximum reported scores over evaluated Ordinary MAS and Graph-enhanced MAS configurations. Each bar selects the highest score within its group and task subset, allowing different configurations to supply different bars.

On the four G-MAS-Complex tiers, the plotted margins are 15, 17, 31, and 45 percentage points. Across all 400 tasks, GoAgent and VeriMap (Chen et al., 2026, Xu et al., 2025) score above the single-agent DeepSeek reference, while the other ten graph-enhanced configurations score below it. Tier maxima describe the best attained scores within each subset. Our Conclusion (C2). Strong complex-task performance is concentrated in a small subset of the evaluated graph-enhanced configurations.

D. Performance Across Task Domains (Q3)

Figure 3 compares four graph-enhanced configurations across six domains.

Code ranges from 53.16% to 57.06%, giving the largest central-score spread of 3.90 percentage points. The spreads are 1.25 on Tabular, 1.06 on Mathematics, 0.53 on Biomedical QA, 0.52 on

Finance, and 0.51 on Knowledge. Appendix A.1 reports dataset-level variation. Our Conclusion (C3). Displayed central-score dispersion varies substantially across task domains.

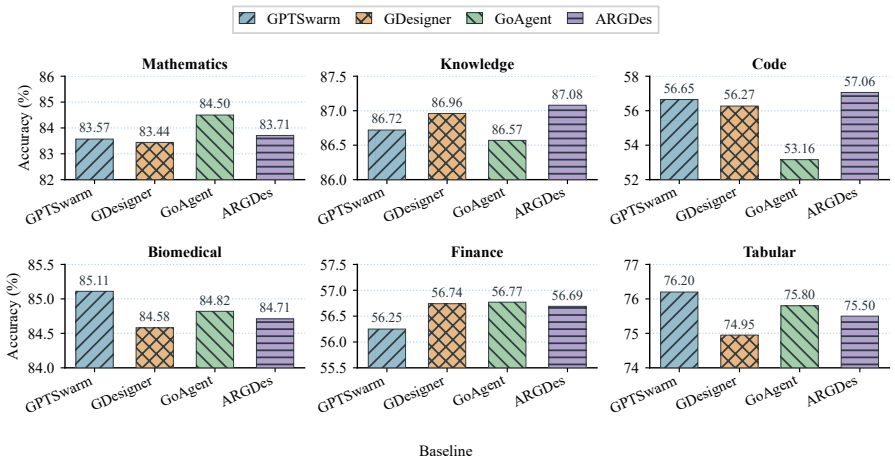


Figure 3. Domain summaries with independent panel scales. Accuracy denotes the reported aggregate score. Appendix A.1 lists dataset-specific metrics.

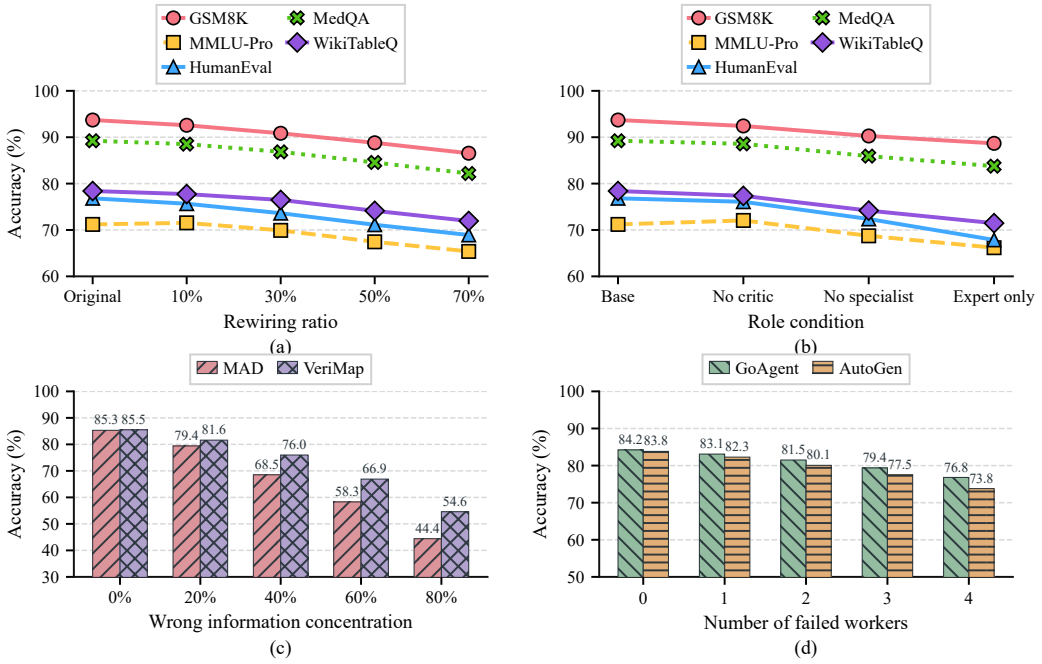


Figure 4. Responses to topology rewiring, role removal, incorrect messages, and worker failures. Panels (a) and (b) show five datasets, while panels (c) and (d) compare the displayed methods. Section 3.3 defines the corresponding operations.

E. Sensitivity to Communication Structure (Q4)

We hold agents, roles, degrees, edge count, models, prompts, routing policy, and budget limits fixed. Rewiring ratios are 10%, 30%, 50%, and 70%. The five-dataset mean falls from 81.88% to 81.21%, 79.54%, 77.21%, and 74.99%. At 70% rewiring, the loss is 6.89 percentage points, leaving 91.59% of the reference score. The progressive decrease associates task performance with

the arrangement of communication partners in the evaluated setting. Actual calls, message counts, and context lengths characterize execution after rewiring. Our Conclusion (C4). Degree-preserving rewiring reduces scores under the specified protocol.

F. Sensitivity to Collaboration-Unit Removal (Q5)

To answer Q5, we compare Base with No critic, No specialist, and Expert only. The first two remove the selected role nodes and incident edges. Expert only keeps the expert-induced subgraph. The mean falls from 81.88% to 81.30%, 78.30%, and 75.59%, respectively, giving losses of 0.58, 3.58, and 6.29 percentage points. Specialist removal incurs 6.17 times the mean loss of critic removal, while MMLU-Pro improves after critic removal. Our Conclusion (C5). Removing different collaboration units produces asymmetric score changes, with the largest loss under Expert only.

G. Robustness to Incorrect Messages (Q6)

We replace eligible upstream records or candidates with format-valid incorrect candidates at corruption rates of 0%, 20%, 40%, 60%, and 80%. The graph and execution controls remain fixed. The concentration axis in Figure 4(c) denotes this item replacement rate. MAD falls from 85.30% to 44.41%, and VeriMap falls from 85.50% to 54.59%. At 80%, the retained scores are 52.06% and 63.85%, respectively. Their accuracy gap grows from 0.20 to 10.18 percentage points. Our Conclusion (C6). Similar reference scores conceal different corruption responses.

H. Recovery Under Worker Failures (Q7)

To answer Q7, we disable 0 to 4 workers and continue through each method's configured retry, fallback, or rerouting procedure. GoAgent decreases from 84.22% to 76.80%, and AutoGen decreases from 83.82% to 73.80%. With four failed workers, the losses are 7.42 and 10.02 percentage points, respectively. The gap grows from 0.40 to 3.00 points across the sweep. This profile compares task completion under the tested failure settings and continuation rules. Our Conclusion (C7). GoAgent retains more reference accuracy than AutoGen in this worker-failure setting.

I. Efficiency (Q8)

For Q8, we compare observed operating points on G-MAS-Complex. Table 2 places Strict EM alongside tokens, latency, and executed nodes and relations. Strict EM/K Tokens divides the strict-match percentage by mean tokens per task in thousands.
Figure 5 ranks observed score-per-token values. AutoGen and VeriMap have the highest ratios, while GoAgent has the highest Strict EM. Compared with DeepSeek, GoAgent gains 23.50 percentage points while using 4.46 times the tokens and 30.11 times the latency. Tokens describe model usage,

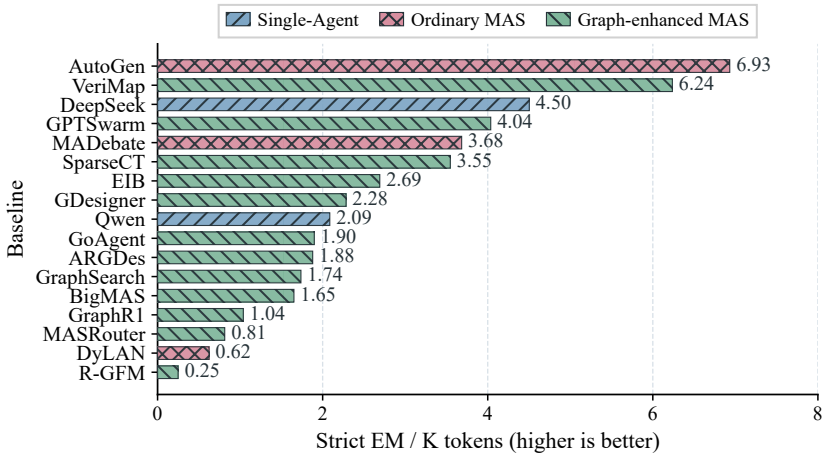


Figure 5. G-MAS-Complex efficiency. Bars show strict exact match per 1K tokens for all baselines.

Table 2. Observed operating points on G-MAS-Complex. Strict EM is in percent, tokens per task in K, and latency in seconds. Nodes and edges are execution averages. Strict EM/K Tokens describes score per 1K tokens at the reported setting.

Method	Strict EM (%)	Tokens/task (K)	Strict EM/K Tokens	Latency (s)	Nodes	Edges
GoAgent	50.25	26.47	1.90	45.46	17.25	67.15
VeriMap	29.50	4.73	6.24	7.09	4.00	4.00
DeepSeek	26.75	5.94	4.50	1.51	1.00	0.00
AutoGen	23.50	3.39	6.93	6.42	3.00	2.00
BigMAS	19.25	11.65	1.65	17.11	14.30	30.90
GPTSwarm	15.50	3.84	4.04	6.48	3.95	2.95
SparseCT	15.25	4.30	3.55	9.25	4.00	3.00
GDesigner	14.50	6.35	2.28	7.95	6.15	9.05
EIB	14.00	5.20	2.69	9.25	6.15	8.05
GraphSearch	14.00	8.06	1.74	11.84	9.15	18.40
MADebate	14.00	3.80	3.68	5.85	4.00	3.00
ARGDes	13.25	7.05	1.88	8.75	7.15	11.25
Qwen	13.25	6.35	2.09	6.41	1.00	0.00
GraphR1	10.75	10.35	1.04	12.60	11.15	28.55
MASRouter	10.75	13.21	0.81	10.76	21.60	38.20
DyLAN	4.50	7.22	0.62	12.67	9.15	12.10
R-GFM	2.25	9.06	0.25	21.27	10.15	13.10

and latency describes elapsed execution under the associated deployment settings. Our Conclusion (C8). The highest accuracy and highest score per token occur at different operating points.

5 Conclusion and Future Directions

OpenMAS-GCom evaluates how graph-enhanced multi-agent systems respond to organizational changes. Its executable representation separates collaboration units, communication relations, shared information, and execution policy. Four paired intervention families modify structure, unit composition, intermediate messages, and worker availability under shared controls. The broad task suite and G-MAS-Complex connect these responses with task outcomes and resource use. The shared interface associates each comparison with its organization settings, intervention parameters, task scores, and execution records.

Three findings organize the evaluated results. First, central-score advantages vary across tasks, and GoAgent and VeriMap are the two graph-enhanced configurations above DeepSeek on G-MAS-Complex. Second, degree-preserving rewiring and collaboration-unit removal produce unequal score changes under their respective settings. At 70% rewiring, the mean score declines by 6.89 points, while specialist removal produces a larger mean decline than critic removal. Third, similar reference scores accompany different responses to incorrect messages and unavailable workers. MAD and VeriMap separate by 10.18 points at 80% corruption, and the accuracy gap between GoAgent and AutoGen increases with worker failures. The configuration with the highest Strict EM also differs from the configuration with the highest score per token.

These findings characterize the tested configurations, tasks, and disruption settings. Paired score changes summarize the response of the complete modified execution. G-MAS-Complex targets document dependencies, record conflicts, and structured completion, while the broad suite retains each dataset's task-specific metric. Resource comparisons describe observed execution settings through scores, token usage, and latency. These views guide configuration selection and organizational diagnosis. Extensions to longer-horizon, interactive, and multimodal tasks can use the same organization interface and paired comparison protocol.

A APPENDIX

This appendix describes datasets, baseline adapters, metrics, interventions, evaluation strategies, execution metadata, parameter settings, supplementary results, and reproducibility records.

A.1 DATASET DESCRIPTION

OpenMAS-GCom contains a broad task suite and a separately constructed G-MAS-Complex stress suite. The broad suite contains 29 datasets across six domains. Mathematics evaluates arithmetic, symbolic, and competition-style reasoning with GSM8K, SVAMP, MultiArith, AQuA, ASDiv-A, Game-of-24, and MATH (Cobbe et al., 2021, Patel et al., 2021, Roy & Roth, 2015, Ling et al., 2017, Miao et al., 2020, Yao et al., 2023, Hendrycks et al., 2021b). Knowledge and commonsense uses MMLU, MMLU-Redux, MMLU-Pro, StrategyQA, CommonsenseQA, ARC-Easy, and ARC-Challenge to test subject knowledge, robust multiple-choice answering, implicit reasoning, and science QA (Hendrycks et al., 2021a, Gema et al., 2025, Wang et al., 2024, Geva et al., 2021, Talmor et al., 2019, Clark et al., 2018). Code uses HumanEval, HumanEval++, LiveCodeBench-v6, and MultiAgentBench-Coding for executable program synthesis and collaborative coding evaluation (Chen et al., 2021a, Liu et al., 2023, Jain et al., 2024, Zhu et al., 2025). Biomedical tasks include PubMedQA, MedQA, MedMCQA, and the medical subset of MMLU (Jin et al., 2019, 2021, Pal et al., 2022, Hendrycks et al., 2021a). Finance uses TAT-QA, ConvFinQA, and FinQA to evaluate numerical reasoning over financial text and tables (Zhu et al., 2021, Chen et al., 2022, 2021b). Tabular reasoning uses TabFact, WikiSQL, WikiTableQuestions, and SQA for table verification, semantic parsing, and sequential table QA (Chen et al., 2020, Zhong et al., 2017, Pasupat & Liang, 2015, Iyyer et al., 2017).

Table 3. BROAD-SUITE DATASET SUMMARY. Domain-level statistics summarize the broad suite. Per-dataset sample counts are shown in Table 11.

Domain	#Datasets	Datasets	Primary evaluation target
Mathematics	7	GSM8K, SVAMP, MultiArith, AQuA, ASDiv-A, Game-of-24, MATH	Numeric answer, multiple-choice answer, valid expression, or competition-style final answer
Knowledge and commonsense	7	MMLU, MMLU-Redux, MMLU-Pro, StrategyQA, CommonsenseQA, ARC-Easy, ARC-Challenge	Multiple-choice or yes/no answer after robust answer extraction
Code	4	HumanEval, HumanEval++, LiveCodeBench-v6, MultiAgentBench-Coding	Unit-test pass rate, executable correctness, or code-task quality score
Biomedical	4	PubMedQA, MedQA, MedMCQA, MMLU-Med	Biomedical yes/no/maybe answer or medical multiple-choice answer
Finance	3	TAT-QA, ConvFinQA, FinQA	Normalized numerical or textual financial answer
Tabular	4	TabFact, WikiSQL, WikiTableQuestions, SQA	Table entailment, table-query answer, or sequential table answer

G-MAS-Complex. We additionally construct 400 frozen multi-document tasks to stress collaboration rather than isolated recall. Each task contains a query, source documents, dependency metadata, conflict annotations, and a machine-checkable JSON output contract. The documents have overlapping contents and distinct functions. Some provide authoritative revisions, some provide intermediate records, some are distractors, and some define constraints that must be combined before the final answer can be produced. This design makes the suite sensitive to delegation, provenance tracking, conflict resolution, and final verification.

For each task, the target answer is represented as

$$z_i = (f_i, v_i, r_i, \ell_i, h_i, \chi_i), \tag{7}$$

Table 4. G-MAS-COMPLEX TIER CONSTRUCTION. Each tier contains 100 frozen tasks.

Tier	Main stress factor	Document and conflict pattern	Required behavior
Medium	Multi-document lookup	Several documents with local dependencies and light distractors	Retrieve relevant records and return a structured answer
Hard	Cross-document dependency	Chained records with competing revisions or aliases	Resolve dependencies before producing fields and references
Ultra	Conflict and provenance	Multiple plausible records, authority rules, and source checkpoints	Select authoritative records and justify structured fields
Extreme	End-to-end audit	Dense dependencies, distractors, revision conflicts, and checksum constraints	Combine retrieval, conflict resolution, ranking, and verification

where f_i contains required fields, v_i contains normalized values, r_i contains ranking or ordering requirements, ℓ_i contains source references, h_i contains checkpoint records, and χ_i contains checksum-style consistency constraints. The output parser first checks whether the response can be mapped to the requested JSON schema, then normalizes scalar values, lists, and source identifiers. A task is counted as correct only when every required component satisfies the contract.

Complex-task validation example. Each stress-suite instance can also be written as $x_i = (q_i, \mathcal{D}_i, \Gamma_i, \Omega_i)$, where q_i is the task query, $\mathcal{D}_i$ is the source-document collection, Γ_i specifies dependencies and record conflicts, and Ω_i defines the output contract. For example, an inventory task may contain three records. D_1 states that 12 units are available, D_2 revises the inventory to 9 units, and D_3 reserves 4 units. The correct answer is therefore 5 available units with source identifiers D_2 and D_3. Using D_1 instead of D_2 yields a content error, while returning the right value without the required source identifiers yields a contract error. The strict scorer rejects both cases because it checks field values, provenance, and schema validity jointly.

A.2 BASELINE DESCRIPTION

Every method is wrapped as an executable organization $\mathcal{O} = (V, E, S, \pi)$ before evaluation. This adapter layer separates method-specific collaboration behavior from shared benchmark components such as dataset loading, backend calls, answer parsing, metric calculation, trace logging, and cost accounting. Single-agent baselines contain one runnable unit and no communication edges. Ordinary MAS baselines contain multiple units with predefined collaboration patterns. Graph-enhanced MAS baselines expose organization graphs, learned or searched routing, sparse communication, verification links, or recovery policies through the same runner interface.

Organization configuration and execution. The configured organization and the realized execution trace are recorded separately. The organization specifies available units, roles, communication relations, routing policy, and recovery policy, while the trace records which units actually execute for a particular input. Execution can differ from the static configuration as messages arrive, candidate answers are rejected, workers become unavailable, or recovery procedures are triggered. This distinction separates available units from executed units and connects the graph specified by a method with the communication actually used to produce a prediction.

Table 5. BASELINE FAMILIES AND ORGANIZATION ADAPTERS.

Family	Methods	Adapter representation
Single-agent	DeepSeek, Qwen (DeepSeek-AI, 2025, Qwen Team, 2025)	One runnable unit, empty edge set, direct answer parser
Ordinary MAS	AutoGen, MADebate, DyLAN (Wu et al., 2023, Du et al., 2023, Liu et al., 2024)	Fixed role set with predefined debate, refinement, or dynamic selection policy
Graph construction and routing	GPTSwarm, GDesigner, GraphSearch, GraphR1, MASRouter (Zhuge et al., 2024, Zhang et al., 2025, Liu et al., 2026b, Luo et al., 2026, Yue et al., 2025)	Explicit graph or search/routing module mapped to typed directed relations
Sparse or role-specialized G-MAS	SparseCT, R-GFM, BigMAS, ARGDes, EIB (Li et al., 2024, Liu et al., 2026a, Hao et al., 2026, Li et al., 2026, Shen et al., 2025)	Specialist units and sparse communication structures mapped to runnable nodes and edges
Verification and recovery-oriented G-MAS	GoAgent, VeriMap (Chen et al., 2026, Xu et al., 2025)	Verification, fallback, or rerouting policies exposed as execution-policy components

A.3 METRIC DESCRIPTION

OpenMAS-GCom reports the official or commonly adopted metric for each dataset whenever possible. For classification and question-answering datasets, the parser maps free-form generations to the answer space before applying exact agreement with the reference. For code tasks, generated programs are evaluated with the corresponding pass@1-style executable test protocol. For tabular and financial reasoning, answers are normalized before comparison, including number formatting, option markers, and short textual aliases when supported by the original benchmark metric.

For a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ and method F, the broad-suite score is

$$s(F, \mathcal{D}) = \frac{1}{n} \sum_{i=1}^{n} m(\mathrm{parse}(F(x_i)), y_i), \tag{8}$$

where m is the dataset-specific metric. For QA datasets, m is accuracy or exact match. For code datasets, m is executable correctness. For MultiAgentBench-Coding, m is CodeQ.

For G-MAS-Complex, strict exact match is

$$\mathrm{StrictEM}(\hat{z}_i, z_i) = \mathbf{1}[\hat{z}_i \models \Omega_i \wedge \hat{z}_i = z_i], \tag{9}$$

and the suite score is

$$s_{\mathrm{complex}}(F) = \frac{1}{400} \sum_{i=1}^{400} \mathrm{StrictEM}(\hat{z}_i, z_i). \tag{10}$$

A prediction is correct only when all required fields, values, rankings, source references, checkpoints, and checksum constraints are satisfied. Partially correct answers remain in the raw prediction logs but receive zero strict exact-match credit.

Efficiency is reported with the same trace fields for all methods. We record token usage, model calls, executed nodes, executed edges, retries, latency, and available cost statistics. For G-MAS-Complex, Table 2 additionally reports $\mathrm{Strict\ EM/K\ Tokens}$, the strict-match percentage per 1K tokens.

A.4 ROBUSTNESS INTERVENTION DESCRIPTION

For every intervention, the task list, backend configuration, prompt template, answer parser, metric, routing policy, and runtime budget remain fixed unless the intervention explicitly targets the

corresponding organization component. Let

$$\mathcal{C} = \{\mathcal{D}_{\mathcal{T}}, \Theta, M_{\mathcal{T}}, P\} \tag{11}$$

denote the shared controls. For intervention type r, we construct

$$\mathcal{O}'_r = \mathcal{I}_r(\mathcal{O}, \Delta\mathcal{O}_r), \qquad r \in \{\text{str}, \text{node}, \text{info}, \text{exec}\}, \tag{12}$$

and measure

$$\Delta_r(F, \mathcal{T}) = s(F, \mathcal{O}'_r, \mathcal{T}) - s(F, \mathcal{O}, \mathcal{T}). \tag{13}$$

A negative value indicates lower performance after modification under the shared controls.

Table 6. CONTROLS PRESERVED BY DIAGNOSTIC INTERVENTIONS. Yes means the factor is kept fixed relative to the reference run.

Intervention	Samples	Backend	Prompts	Parser	Budget	Roles	Edges	Degree seq.
Structure rewiring	Yes	Yes	Yes	Yes	Yes	Yes	Changed	Yes
Node removal	Yes	Yes	Yes	Yes	Yes	Changed	Changed	No
Information injection	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Execution failure	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Structure intervention. We evaluate the Original graph and degree-preserving rewiring ratios of 10%, 30%, 50%, and 70%. Directed edge pairs $u \to a$ and $v \to b$ are proposed for the swap $u \to b$ and $v \to a$. Proposals that create self-loops, duplicate edges, or invalid role connections are rejected. The actual ratio and random seed are recorded.

Node intervention. The No critic and No specialist variants remove the corresponding role nodes and all incident edges. The Expert only variant retains expert nodes and their induced communication edges. This intervention changes collaboration-unit composition while keeping the task, model, prompt, parser, routing, and budget controls fixed.

Information intervention. For concentration $c \in \{0, .2, .4, .6, .8\}$, a fraction c of eligible upstream records or candidate items is replaced with a format-valid but incorrect candidate and forwarded through the unchanged downstream graph. We report task accuracy and whether final answers follow the injected candidate. The trace further records whether the injected candidate reaches downstream units and whether it is adopted, corrected, or ignored before the final answer.

Execution intervention. For each failure count $k \in \{0, 1, 2, 3, 4\}$, k workers are marked unavailable at selected execution points. The last valid shared state is recorded and the method resumes through its configured retry, fallback, or rerouting mechanism. $k = 0$ denotes no failure.

A.5 EVALUATION STRATEGIES

The benchmark follows four evaluation perspectives, matching the main questions in Section 4. Necessity compares single-agent, ordinary MAS, and graph-enhanced MAS under identical task and metric settings. Effectiveness compares method behavior across the six broad-suite domains and the four G-MAS-Complex tiers. Robustness applies the four controlled interventions above to isolate communication topology, role composition, information quality, and execution recovery. Efficiency reports quality together with token usage, model calls, latency, executed nodes and edges, and retries.

A.6 EXPERIMENT ENVIRONMENT

The execution record associates each run with its task, organization, backend configuration, and resource measurements. All methods are executed through the same runner interface. Each run records the dataset identifier, sample identifier, backend configuration, prompt-template hash, organization hash, final prediction, parsed prediction, metric result, token usage, model calls, executed nodes and edges, retries, latency, and termination status. Intervention runs additionally record the reference organization hash, intervention type, intervention parameters, changed nodes or edges when applicable, injected-record identifiers, failed-worker identifiers, and selected recovery action.

Table 7. EVALUATION STRATEGY SUMMARY.

Perspective	Main comparison	Reported results
Necessity	Single-agent vs. ordinary MAS vs. graph-enhanced MAS	Main 13-dataset comparison and full 29-dataset table
Effectiveness	Method behavior across domains and complex-task tiers	Domain averages, G-MAS-Complex tiers, and complete method rankings
Robustness	Structure, node, information, and execution interventions	Performance changes under controlled organizational perturbations
Efficiency	Task quality relative to execution cost	Strict EM, tokens, latency, executed nodes, executed edges, and Strict EM/K tokens

Table 8. RECORDED ENVIRONMENT AND RUN METADATA. The record links task, organization, backend, prediction, cost, and intervention fields.

Category	Recorded fields
Task state	Dataset id, sample id, split or frozen-list identifier, gold answer, parser version
Organization state	Method id, organization hash, node list, edge list, role configuration, routing policy
Backend state	Backend configuration id, prompt-template hash, runtime limits, random seed when used
Prediction state	Raw response, parsed prediction, invalid-parse flag, metric result, final answer
Cost state	Token usage, model calls, executed nodes, executed edges, retries, latency, termination reason
Intervention state	Reference organization hash, intervention type, changed nodes/edges, injected records, failed workers, recovery action

Before aggregation, the audit script checks that reference and intervention runs share the same task identifiers, parser version, backend configuration, prompt template, and stopping budget. Runs that fail these consistency checks are excluded from aggregate tables until regenerated.

Validation protocol. Each run produces a validation record before it enters an aggregate table. The record includes the dataset and sample identifiers, method configuration, parser version, raw answer, parsed answer, metric value, and trace hash. For G-MAS-Complex, the validator additionally distinguishes output-format failures from content failures. Format failures cannot be mapped to Ω_i, while content failures satisfy the schema but miss required values, rankings, references, checkpoints, or checksum constraints. This separation keeps structured-output reliability visible instead of folding every failure into a single exact-match score.

A.7 HYPERPARAMETER SETTINGS

All comparisons freeze task samples, prompt templates, answer parsers, backend configuration, routing policy, and runtime limits within a comparison track. The broad suite follows fixed dataset-specific evaluation lists. Datasets with repeated-sampling analysis use three matched 500-instance samples where applicable. G-MAS-Complex is fixed. Every method is evaluated on the same 400 frozen tasks. Intervention sweeps use the discrete settings in Table 9. No intervention changes the task text, gold answer, scoring parser, or input ordering.

A.8 UNCERTAINTY SOURCES

The detailed tables distinguish five statistical sources through cell-level codes. Table 10 records the quantity attached to each score and the available repetition information. Standard deviations, analytic standard errors, and empirical estimates retain their distinct statistical meanings. Rank-

Table 9. BENCHMARK-LEVEL HYPERPARAMETER AND SWEEP SETTINGS.

Component	Setting
Broad-suite sampling	Full evaluation split where used, otherwise fixed ordered subset, with three matched 500-instance samples for repeated-sampling datasets
G-MAS-Complex sampling	400 frozen tasks, 100 tasks each for Medium, Hard, Ultra, and Extreme
Structure rewiring	Original, 10%, 30%, 50%, and 70% degree-preserving rewiring
Node intervention	Base, No critic, No specialist, Expert only
Information intervention	Incorrect-information concentration $c \in \{0, .2, .4, .6, .8\}$
Execution intervention	Failed workers $k \in \{0, 1, 2, 3, 4\}$
Uncertainty reporting	Three-sample standard deviation where available, otherwise run-seed standard deviation, recorded estimate, or binomial standard-error proxy

ing markers order the displayed central scores, including equal numerical values. A paired score comparison specifies the common statistic and the matched task and seed records.

Code	Cells	Recorded quantity	Repetition information
s	221	Mean and sample standard deviation across sampled task sets. Each set contains 500 instances.	Three sample sets
r	152	Fixed reference score with an attached run-seed standard deviation.	Run count unrecorded
e	34	Reference score with a recorded empirical estimate combining dataset and method components at weights 0.70 and 0.30.	Component counts unrecorded
b	100	Reference score with a binomial standard-error estimate computed from that score and the displayed evaluation count N.	Analytic estimate
m	3	Backup run-seed mean and standard deviation for cells with an empty reference metric.	Run count unrecorded

Table 10. Statistical sources and cell counts for the 510 entries in the detailed result tables.

For the three percentage scores y_1, y_2, y_3, code s uses

$$\bar{y} = \frac{1}{3}\sum_{j=1}^{3} y_j, \qquad s = \sqrt{\frac{1}{2}\sum_{j=1}^{3}(y_j - \bar{y})^2}. \tag{14}$$

For code b, let p be the reported score divided by 100. The recorded estimate in percentage points is $100\sqrt{p(1-p)/N}$, using an independent binary-outcome model. Applying this model to CodeQ requires a binary task-outcome definition in the scoring record. The three code m entries are DeepSeek, BigMAS, and MASRouter on MultiAgentBench-Coding. Source records preserve the central value and the statistical quantity as separate fields.

A.9 SUPPLEMENTARY RESULTS

The complete 29-dataset and 17-baseline comparison is reported in Table 11. The repeated-sampling summary for the 13 matched-sampling datasets is provided in Table 18. These tables are separated from the dataset description to keep the appendix layout stable.

Reading the detailed results. Each entry reports a central score in percent, followed by an uncertainty term in percentage points and a source code. Code s gives the mean and sample standard deviation across three 500-instance sample sets. Code r appends a recorded run-seed standard deviation to the fixed reference score. Code e appends the recorded estimate combining dataset and method components with weights 0.70 and 0.30. Code b appends a binomial standard-error estimate

using the displayed N. Code m gives a backup run-seed mean and standard deviation for the three cells whose reference metric is missing. Bold and underlined scores mark the highest and second-highest distinct values within each dataset, including ties. Horizontal rules separate single-agent, ordinary multi-agent, and graph-enhanced configurations.

Method	GSM8K Acc. $N = 1319$	SVAMP Acc. $N = 1000$	MultiArith Acc. $N = 600$	AQuA Acc. $N = 254$	ASDiv-A Acc. $N = 500$	Game-of-24 Acc. $N = 1362$	MATH Acc. $N = 500$
DeepSeek	$\underline{93.71}$ $(0.33)^{\mathrm{r}}$	94.00 $(0.10)^{\mathrm{r}}$	98.50 $(0.25)^{\mathrm{r}}$	89.76 $(0.45)^{\mathrm{r}}$	88.53 $(1.10)^{\mathrm{s}}$	58.88 $(1.33)^{\mathrm{b}}$	55.73 $(3.64)^{\mathrm{s}}$
Qwen	59.44 $(0.56)^{\mathrm{e}}$	90.90 $(0.91)^{\mathrm{b}}$	97.17 $(0.68)^{\mathrm{b}}$	68.90 $(0.86)^{\mathrm{e}}$	86.87 $(0.42)^{\mathrm{s}}$	21.00 $(1.10)^{\mathrm{b}}$	45.20 $(2.27)^{\mathrm{s}}$
AutoGen	92.87 $(0.34)^{\mathrm{r}}$	93.70 $(0.77)^{\mathrm{b}}$	98.50 $(0.19)^{\mathrm{r}}$	88.98 $(1.72)^{\mathrm{r}}$	88.53 $(0.99)^{\mathrm{s}}$	$\underline{67.91}$ $(1.26)^{\mathrm{b}}$	56.13 $(2.66)^{\mathrm{s}}$
MADebate	92.87 $(0.24)^{\mathrm{r}}$	94.20 $(0.74)^{\mathrm{b}}$	$\mathbf{98.83}$ $(0.00)^{\mathrm{r}}$	89.37 $(0.23)^{\mathrm{r}}$	88.53 $(0.99)^{\mathrm{s}}$	62.70 $(1.31)^{\mathrm{b}}$	55.27 $(3.42)^{\mathrm{s}}$
DyLAN	93.33 $(0.41)^{\mathrm{e}}$	94.00 $(0.25)^{\mathrm{r}}$	$\underline{98.67}$ $(0.29)^{\mathrm{r}}$	89.76 $(0.99)^{\mathrm{r}}$	$\mathbf{88.80}$ $(1.40)^{\mathrm{s}}$	62.41 $(1.31)^{\mathrm{b}}$	56.07 $(3.24)^{\mathrm{s}}$
GPTSwarm	93.56 $(0.43)^{\mathrm{r}}$	94.10 $(0.46)^{\mathrm{r}}$	98.50 $(0.25)^{\mathrm{r}}$	$\mathbf{91.34}$ $(0.68)^{\mathrm{e}}$	88.67 $(0.81)^{\mathrm{s}}$	65.71 $(0.85)^{\mathrm{r}}$	57.00 $(2.84)^{\mathrm{s}}$
GDesigner	93.63 $(0.27)^{\mathrm{r}}$	94.00 $(0.06)^{\mathrm{r}}$	$\mathbf{98.83}$ $(0.10)^{\mathrm{r}}$	88.98 $(0.82)^{\mathrm{r}}$	88.60 $(1.31)^{\mathrm{s}}$	67.62 $(1.27)^{\mathrm{b}}$	56.73 $(3.37)^{\mathrm{s}}$
SparseCT	$\mathbf{93.86}$ $(0.41)^{\mathrm{e}}$	93.90 $(0.76)^{\mathrm{b}}$	$\underline{98.67}$ $(0.10)^{\mathrm{r}}$	89.76 $(0.68)^{\mathrm{r}}$	$\mathbf{88.80}$ $(0.92)^{\mathrm{s}}$	64.54 $(1.30)^{\mathrm{b}}$	$\underline{57.13}$ $(3.87)^{\mathrm{s}}$
GraphSearch	93.56 $(0.13)^{\mathrm{r}}$	94.10 $(0.21)^{\mathrm{r}}$	$\underline{98.67}$ $(0.00)^{\mathrm{r}}$	89.37 $(1.20)^{\mathrm{r}}$	88.67 $(0.90)^{\mathrm{s}}$	67.55 $(1.27)^{\mathrm{b}}$	56.40 $(3.46)^{\mathrm{s}}$
GraphR1	92.87 $(0.27)^{\mathrm{r}}$	93.60 $(0.55)^{\mathrm{r}}$	98.50 $(0.10)^{\mathrm{r}}$	$\underline{90.55}$ $(0.23)^{\mathrm{r}}$	88.60 $(1.06)^{\mathrm{s}}$	66.89 $(1.28)^{\mathrm{b}}$	56.73 $(2.72)^{\mathrm{s}}$
R-GFM	93.18 $(0.20)^{\mathrm{r}}$	93.80 $(0.26)^{\mathrm{r}}$	98.50 $(0.10)^{\mathrm{r}}$	89.76 $(0.91)^{\mathrm{r}}$	$\mathbf{88.80}$ $(1.44)^{\mathrm{s}}$	66.08 $(1.28)^{\mathrm{b}}$	55.73 $(3.88)^{\mathrm{s}}$
BigMAS	93.18 $(0.19)^{\mathrm{r}}$	$\underline{94.30}$ $(0.44)^{\mathrm{r}}$	$\underline{98.67}$ $(0.10)^{\mathrm{r}}$	$\underline{90.55}$ $(0.45)^{\mathrm{r}}$	88.53 $(1.51)^{\mathrm{s}}$	65.93 $(1.28)^{\mathrm{b}}$	$\mathbf{57.47}$ $(3.59)^{\mathrm{s}}$
MASRouter	$\underline{93.71}$ $(0.23)^{\mathrm{r}}$	93.80 $(0.17)^{\mathrm{r}}$	98.50 $(0.10)^{\mathrm{r}}$	88.98 $(0.39)^{\mathrm{r}}$	88.40 $(1.59)^{\mathrm{s}}$	63.14 $(1.31)^{\mathrm{b}}$	56.00 $(3.33)^{\mathrm{s}}$
GoAgent	92.87 $(0.04)^{\mathrm{r}}$	$\underline{94.30}$ $(0.31)^{\mathrm{r}}$	98.50 $(0.00)^{\mathrm{r}}$	89.76 $(0.79)^{\mathrm{r}}$	88.53 $(0.99)^{\mathrm{s}}$	$\mathbf{74.08}$ $(1.19)^{\mathrm{b}}$	56.07 $(2.89)^{\mathrm{s}}$
VeriMap	93.63 $(0.39)^{\mathrm{r}}$	93.60 $(0.25)^{\mathrm{r}}$	$\mathbf{98.83}$ $(0.10)^{\mathrm{r}}$	89.76 $(0.68)^{\mathrm{r}}$	$\underline{88.73}$ $(0.50)^{\mathrm{s}}$	65.71 $(0.28)^{\mathrm{r}}$	56.33 $(3.87)^{\mathrm{s}}$
ARGDes	93.63 $(0.38)^{\mathrm{r}}$	$\mathbf{94.60}$ $(0.71)^{\mathrm{b}}$	$\underline{98.67}$ $(0.17)^{\mathrm{r}}$	$\underline{90.55}$ $(0.99)^{\mathrm{r}}$	88.53 $(1.17)^{\mathrm{s}}$	67.11 $(1.27)^{\mathrm{b}}$	56.53 $(3.01)^{\mathrm{s}}$
EIB	93.56 $(0.08)^{\mathrm{r}}$	$\underline{94.30}$ $(0.10)^{\mathrm{r}}$	98.50 $(0.10)^{\mathrm{r}}$	89.76 $(0.39)^{\mathrm{r}}$	$\mathbf{88.80}$ $(1.40)^{\mathrm{s}}$	67.84 $(1.27)^{\mathrm{b}}$	56.60 $(3.29)^{\mathrm{s}}$

Table 11. Mathematical reasoning results. Each entry places the central score above its uncertainty term and source code. Source codes are defined above.

Method	MMLU Acc. $N = 500$	MMLU-Redux Acc. $N = 500$	MMLU-Pro Acc. $N = 500$	StrategyQA Acc. $N = 500$	Commonsense QA Acc. $N = 500$	ARC-Easy Acc. $N = 500$	ARC-Challenge Acc. $N = 1418$
DeepSeek	85.40 $(0.92)^{s}$	83.73 $(0.50)^{s}$	73.07 $(0.76)^{s}$	84.60 $(0.80)^{s}$	85.40 $(0.53)^{s}$	<u>94.27</u> $(0.61)^{s}$	96.26 $(0.50)^{b}$
Qwen	82.33 $(2.76)^{s}$	79.80 $(0.92)^{s}$	60.93 $(0.90)^{s}$	79.40 $(1.93)^{s}$	85.67 $(1.33)^{s}$	94.07 $(0.76)^{s}$	96.05 $(0.52)^{b}$
AutoGen	85.80 $(0.35)^{s}$	84.00 $(1.73)^{s}$	71.80 $(1.20)^{s}$	84.13 $(0.70)^{s}$	85.67 $(1.30)^{s}$	<u>94.27</u> $(0.76)^{s}$	96.61 $(0.15)^{r}$
MADebate	85.93 $(0.12)^{s}$	83.60 $(0.53)^{s}$	71.87 $(1.14)^{s}$	84.07 $(1.96)^{s}$	85.20 $(1.56)^{s}$	94.07 $(0.81)^{s}$	96.54 $(0.49)^{b}$
DyLAN	86.00 $(1.39)^{s}$	83.93 $(1.50)^{s}$	**73.87** $(1.21)^{s}$	**86.67** $(0.70)^{s}$	85.73 $(0.61)^{s}$	94.07 $(0.76)^{s}$	<u>96.76</u> $(0.19)^{r}$
GPTSwarm	85.60 $(0.53)^{s}$	<u>84.40</u> $(0.35)^{s}$	70.73 $(0.61)^{s}$	85.67 $(2.84)^{s}$	85.53 $(0.64)^{s}$	<u>94.27</u> $(0.76)^{s}$	**96.83** $(0.19)^{r}$
GDesigner	85.67 $(1.51)^{s}$	84.27 $(1.17)^{s}$	71.87 $(0.31)^{s}$	85.00 $(2.03)^{s}$	86.13 $(1.36)^{s}$	94.20 $(1.06)^{s}$	96.33 $(0.50)^{b}$
SparseCT	85.20 $(0.87)^{s}$	83.73 $(0.50)^{s}$	71.33 $(1.10)^{s}$	85.80 $(0.80)^{s}$	86.13 $(1.21)^{s}$	93.93 $(0.64)^{s}$	96.69 $(0.48)^{b}$
GraphSearch	85.80 $(0.80)^{s}$	<u>84.40</u> $(1.91)^{s}$	71.60 $(0.92)^{s}$	85.60 $(0.20)^{s}$	86.07 $(1.27)^{s}$	<u>94.27</u> $(0.58)^{s}$	96.40 $(0.49)^{b}$
GraphR1	85.33 $(1.30)^{s}$	84.00 $(1.06)^{s}$	70.53 $(1.33)^{s}$	85.60 $(0.69)^{s}$	85.47 $(1.67)^{s}$	94.20 $(0.72)^{s}$	**96.83** $(0.18)^{r}$
R-GFM	85.47 $(1.27)^{s}$	84.27 $(1.62)^{s}$	71.53 $(0.50)^{s}$	85.13 $(0.50)^{s}$	85.93 $(1.30)^{s}$	93.93 $(0.83)^{s}$	<u>96.76</u> $(0.36)^{r}$
BigMAS	86.00 $(1.11)^{s}$	<u>84.40</u> $(0.92)^{s}$	72.20 $(0.20)^{s}$	85.07 $(1.62)^{s}$	85.87 $(1.03)^{s}$	93.93 $(1.03)^{s}$	**96.83** $(0.16)^{r}$
MASRouter	<u>86.53</u> $(1.03)^{s}$	84.33 $(0.83)^{s}$	72.53 $(1.81)^{s}$	85.67 $(0.58)^{s}$	86.07 $(0.70)^{s}$	**94.33** $(0.64)^{s}$	96.61 $(0.25)^{r}$
GoAgent	85.93 $(1.14)^{s}$	83.87 $(1.33)^{s}$	70.33 $(0.50)^{s}$	85.27 $(2.05)^{s}$	**86.47** $(1.45)^{s}$	94.00 $(1.06)^{s}$	<u>96.76</u> $(0.18)^{r}$
VeriMap	85.87 $(0.46)^{s}$	84.13 $(1.51)^{s}$	<u>73.53</u> $(1.70)^{s}$	83.53 $(1.17)^{s}$	85.40 $(1.00)^{s}$	93.87 $(0.92)^{s}$	<u>96.76</u> $(0.12)^{r}$
ARGDes	**87.27** $(1.10)^{s}$	83.60 $(1.04)^{s}$	71.47 $(0.58)^{s}$	<u>86.00</u> $(1.59)^{s}$	<u>86.20</u> $(2.31)^{s}$	94.13 $(0.64)^{s}$	<u>96.76</u> $(0.25)^{r}$
EIB	86.13 $(1.67)^{s}$	**84.53** $(1.42)^{s}$	72.47 $(0.12)^{s}$	85.40 $(1.40)^{s}$	85.73 $(1.17)^{s}$	93.93 $(0.70)^{s}$	96.61 $(0.22)^{r}$

Table 12. Knowledge and commonsense results. Each entry places the central score above its uncertainty term and source code. Source codes are defined above.

Method	HumanEval Acc. $N = 164$	HumanEval++ Acc. $N = 164$	LiveCodeBench v6 Pass@1 $N = 175$	MultiAgentBench Coding CodeQ $N = 102$
DeepSeek	$84.76\,(0.00)^{\mathrm{r}}$	$\mathbf{80.49}\,(2.14)^{\mathrm{r}}$	$20.57\,(2.01)^{\mathrm{r}}$	$41.55\,(1.06)^{\mathrm{m}}$
Qwen	$\mathbf{90.24}\,(1.35)^{\mathrm{e}}$	$76.83\,(0.70)^{\mathrm{r}}$	$\mathbf{26.86}\,(1.39)^{\mathrm{e}}$	$\mathbf{60.54}\,(4.84)^{\mathrm{b}}$
AutoGen	$81.71\,(1.21)^{\mathrm{e}}$	$73.78\,(3.43)^{\mathrm{b}}$	$18.86\,(1.25)^{\mathrm{e}}$	$42.01\,(4.89)^{\mathrm{b}}$
MADebate	$83.54\,(2.11)^{\mathrm{r}}$	$77.44\,(2.46)^{\mathrm{r}}$	$21.71\,(2.88)^{\mathrm{r}}$	$40.69\,(4.86)^{\mathrm{b}}$
DyLAN	$85.98\,(1.19)^{\mathrm{e}}$	$78.66\,(3.20)^{\mathrm{b}}$	$22.86\,(1.23)^{\mathrm{e}}$	$37.06\,(4.78)^{\mathrm{b}}$
GPTSwarm	$84.76\,(0.93)^{\mathrm{r}}$	$\mathbf{80.49}\,(5.21)^{\mathrm{r}}$	$19.43\,(0.00)^{\mathrm{r}}$	$41.91\,(4.89)^{\mathrm{b}}$
GDesigner	$85.37\,(1.21)^{\mathrm{e}}$	$76.83\,(1.76)^{\mathrm{r}}$	$\underline{23.43}\,(1.25)^{\mathrm{e}}$	$39.46\,(4.84)^{\mathrm{b}}$
SparseCT	$84.76\,(1.83)^{\mathrm{r}}$	$76.83\,(1.22)^{\mathrm{r}}$	$21.14\,(1.44)^{\mathrm{r}}$	$\underline{43.82}\,(4.91)^{\mathrm{b}}$
GraphSearch	$81.10\,(3.06)^{\mathrm{b}}$	$74.39\,(1.06)^{\mathrm{r}}$	$22.29\,(3.15)^{\mathrm{b}}$	$40.64\,(4.86)^{\mathrm{b}}$
GraphR1	$86.59\,(2.66)^{\mathrm{b}}$	$75.61\,(1.06)^{\mathrm{r}}$	$20.00\,(3.02)^{\mathrm{b}}$	$38.09\,(4.81)^{\mathrm{b}}$
R-GFM	$89.02\,(1.16)^{\mathrm{e}}$	$75.61\,(2.54)^{\mathrm{r}}$	$19.43\,(0.87)^{\mathrm{r}}$	$40.74\,(4.87)^{\mathrm{b}}$
BigMAS	$84.76\,(0.35)^{\mathrm{r}}$	$76.22\,(3.32)^{\mathrm{b}}$	$21.71\,(1.44)^{\mathrm{r}}$	$37.79\,(1.04)^{\mathrm{m}}$
MASRouter	$76.83\,(3.29)^{\mathrm{b}}$	$71.95\,(3.51)^{\mathrm{b}}$	$20.57\,(0.57)^{\mathrm{r}}$	$37.45\,(2.48)^{\mathrm{m}}$
GoAgent	$79.88\,(1.15)^{\mathrm{e}}$	$74.39\,(2.46)^{\mathrm{r}}$	$20.57\,(1.51)^{\mathrm{r}}$	$37.79\,(4.80)^{\mathrm{b}}$
VeriMap	$78.66\,(3.20)^{\mathrm{b}}$	$77.44\,(3.66)^{\mathrm{r}}$	$21.14\,(0.33)^{\mathrm{r}}$	$40.15\,(4.85)^{\mathrm{b}}$
ARGDes	$\underline{89.63}\,(1.18)^{\mathrm{e}}$	$\underline{79.27}\,(3.17)^{\mathrm{b}}$	$19.43\,(1.84)^{\mathrm{r}}$	$39.90\,(4.85)^{\mathrm{b}}$
EIB	$83.54\,(2.20)^{\mathrm{r}}$	$76.83\,(1.27)^{\mathrm{r}}$	$21.71\,(1.44)^{\mathrm{r}}$	$42.35\,(4.89)^{\mathrm{b}}$

Table 13. Code generation results. Entries give score (uncertainty)^{source} with the defined source codes. The recorded CodeQ metric is retained for MultiAgentBench-Coding.

Method	PubMedQA Acc. $N = 3000$	MedQA Acc. $N = 1273$	MedMCQA Acc. $N = 500$	MMLU-Med Acc. $N = 1561$
DeepSeek	$81.33\,(0.71)^{\mathrm{b}}$	$88.45\,(0.68)^{\mathrm{r}}$	$79.67\,(0.99)^{\mathrm{s}}$	$89.75\,(0.77)^{\mathrm{b}}$
Qwen	$80.70\,(1.25)^{\mathrm{b}}$	$78.95\,(0.72)^{\mathrm{e}}$	$73.53\,(0.61)^{\mathrm{s}}$	$87.51\,(0.84)^{\mathrm{b}}$
AutoGen	$81.43\,(0.71)^{\mathrm{b}}$	$89.08\,(0.55)^{\mathrm{r}}$	$79.67\,(1.70)^{\mathrm{s}}$	$89.94\,(0.15)^{\mathrm{r}}$
MADebate	$81.87\,(0.70)^{\mathrm{b}}$	$89.08\,(0.53)^{\mathrm{r}}$	$78.53\,(1.14)^{\mathrm{s}}$	$\underline{90.26}\,(0.75)^{\mathrm{b}}$
DyLAN	$\mathbf{82.43}\,(0.69)^{\mathrm{b}}$	$88.77\,(0.08)^{\mathrm{r}}$	$79.93\,(1.10)^{\mathrm{s}}$	$89.56\,(0.38)^{\mathrm{r}}$
GPTSwarm	$81.67\,(0.71)^{\mathrm{b}}$	$89.63\,(0.54)^{\mathrm{e}}$	$79.00\,(0.87)^{\mathrm{s}}$	$89.94\,(0.24)^{\mathrm{r}}$
GDesigner	$81.37\,(0.71)^{\mathrm{b}}$	$88.45\,(0.48)^{\mathrm{r}}$	$79.93\,(0.76)^{\mathrm{s}}$	$89.69\,(0.77)^{\mathrm{b}}$
SparseCT	$81.60\,(0.67)^{\mathrm{r}}$	$89.24\,(0.21)^{\mathrm{r}}$	$79.67\,(1.67)^{\mathrm{s}}$	$89.88\,(0.39)^{\mathrm{r}}$
GraphSearch	$81.53\,(0.44)^{\mathrm{r}}$	$\underline{89.71}\,(0.85)^{\mathrm{b}}$	$78.73\,(1.01)^{\mathrm{s}}$	$89.88\,(0.27)^{\mathrm{r}}$
GraphR1	$81.30\,(0.71)^{\mathrm{b}}$	$88.92\,(0.88)^{\mathrm{b}}$	$79.13\,(0.83)^{\mathrm{s}}$	$\mathbf{90.39}\,(0.75)^{\mathrm{b}}$
R-GFM	$81.53\,(0.21)^{\mathrm{r}}$	$89.16\,(0.70)^{\mathrm{r}}$	$79.53\,(0.95)^{\mathrm{s}}$	$89.88\,(0.13)^{\mathrm{r}}$
BigMAS	$81.20\,(0.30)^{\mathrm{r}}$	$\mathbf{89.87}\,(0.53)^{\mathrm{e}}$	$78.73\,(1.85)^{\mathrm{s}}$	$90.07\,(0.19)^{\mathrm{r}}$
MASRouter	$81.60\,(0.06)^{\mathrm{r}}$	$89.24\,(0.33)^{\mathrm{r}}$	$78.80\,(1.91)^{\mathrm{s}}$	$89.56\,(0.10)^{\mathrm{r}}$
GoAgent	$80.53\,(0.72)^{\mathrm{b}}$	$89.40\,(0.51)^{\mathrm{e}}$	$79.13\,(0.92)^{\mathrm{s}}$	$89.56\,(0.33)^{\mathrm{r}}$
VeriMap	$81.27\,(0.71)^{\mathrm{b}}$	$88.77\,(0.52)^{\mathrm{r}}$	$\underline{80.07}\,(1.51)^{\mathrm{s}}$	$90.07\,(0.11)^{\mathrm{r}}$
ARGDes	$\underline{82.03}\,(0.70)^{\mathrm{b}}$	$88.77\,(0.42)^{\mathrm{r}}$	$\mathbf{80.53}\,(0.70)^{\mathrm{s}}$	$89.62\,(0.77)^{\mathrm{b}}$
EIB	$81.20\,(0.46)^{\mathrm{r}}$	$89.16\,(0.48)^{\mathrm{r}}$	$78.87\,(1.33)^{\mathrm{s}}$	$90.20\,(0.75)^{\mathrm{b}}$

Table 14. Biomedical reasoning results. Entries give score (uncertainty)^{source} with the defined source codes. PubMedQA retains the recorded source sample count.

Method	TAT-QA Acc. $N = 1663$	ConvFinQA Acc. $N = 1490$	FinQA Acc. $N = 1147$
DeepSeek	70.48 (0.63)^{r}	60.13 (0.43)^{e}	38.10 (0.51)^{e}
Qwen	69.51 (1.13)^{b}	**61.81** (0.62)^{e}	28.51 (0.70)^{e}
AutoGen	70.29 (1.12)^{b}	60.74 (0.66)^{r}	36.70 (0.57)^{e}
MADebate	70.66 (1.12)^{b}	61.01 (0.52)^{e}	36.97 (0.61)^{e}
DyLAN	70.48 (0.23)^{r}	60.60 (0.55)^{r}	38.88 (0.55)^{e}
GPTSwarm	70.41 (0.33)^{r}	60.67 (0.24)^{r}	37.66 (1.13)^{r}
GDesigner	70.23 (0.44)^{r}	61.54 (0.48)^{e}	38.45 (1.02)^{r}
SparseCT	70.41 (0.38)^{r}	60.81 (0.47)^{e}	**39.23** (0.56)^{e}
GraphSearch	**71.26** (1.11)^{b}	60.20 (0.49)^{r}	38.45 (0.18)^{r}
GraphR1	69.75 (1.13)^{b}	60.67 (0.34)^{r}	38.01 (1.43)^{b}
R-GFM	71.20 (1.11)^{b}	60.40 (0.24)^{r}	38.45 (0.46)^{r}
BigMAS	70.41 (0.68)^{r}	60.40 (0.21)^{r}	38.36 (0.52)^{e}
MASRouter	69.99 (0.24)^{r}	60.20 (0.34)^{r}	37.14 (1.43)^{b}
GoAgent	71.14 (1.11)^{b}	60.74 (0.47)^{r}	38.45 (0.13)^{r}
VeriMap	70.23 (0.33)^{r}	61.28 (1.26)^{b}	37.66 (0.83)^{r}
ARGDes	71.02 (1.11)^{b}	60.60 (0.12)^{r}	38.45 (0.31)^{r}
EIB	69.99 (0.67)^{r}	61.34 (0.45)^{e}	38.54 (0.54)^{e}

Table 15. Financial reasoning results with uncertainty terms and source codes.

Method	TabFact Acc. $N = 500$	WikiSQL Acc. $N = 500$	WikiTable Questions Acc. $N = 500$	SQA Acc. $N = 500$
DeepSeek	92.00 (1.11)^{s}	74.07 (0.61)^{s}	76.73 (2.16)^{s}	55.20 (1.44)^{s}
Qwen	81.40 (1.56)^{s}	74.93 (1.27)^{s}	65.27 (1.50)^{s}	46.53 (2.14)^{s}
AutoGen	91.80 (1.22)^{s}	74.20 (1.78)^{s}	75.93 (1.03)^{s}	54.67 (1.97)^{s}
MADebate	91.47 (0.42)^{s}	74.27 (1.10)^{s}	76.67 (1.33)^{s}	54.60 (1.93)^{s}
DyLAN	**92.53** (1.53)^{s}	73.87 (2.53)^{s}	77.40 (1.44)^{s}	55.00 (1.74)^{s}
GPTSwarm	92.20 (1.40)^{s}	74.80 (1.97)^{s}	76.60 (1.93)^{s}	56.13 (3.13)^{s}
GDesigner	91.73 (0.61)^{s}	74.93 (0.83)^{s}	76.60 (1.06)^{s}	55.73 (4.12)^{s}
SparseCT	91.93 (1.10)^{s}	74.87 (1.22)^{s}	**77.47** (0.76)^{s}	56.53 (1.86)^{s}
GraphSearch	92.00 (1.56)^{s}	74.67 (0.81)^{s}	77.13 (1.62)^{s}	55.60 (1.44)^{s}
GraphR1	91.40 (1.71)^{s}	74.40 (0.69)^{s}	76.80 (0.20)^{s}	56.13 (1.81)^{s}
R-GFM	92.13 (2.30)^{s}	**75.60** (1.25)^{s}	77.40 (1.56)^{s}	56.07 (3.67)^{s}
BigMAS	92.33 (1.29)^{s}	74.73 (0.90)^{s}	76.80 (1.11)^{s}	55.33 (3.03)^{s}
MASRouter	91.40 (1.11)^{s}	74.00 (2.55)^{s}	75.67 (0.81)^{s}	57.60 (1.51)^{s}
GoAgent	92.33 (0.81)^{s}	74.47 (1.79)^{s}	76.20 (0.80)^{s}	**57.73** (2.34)^{s}
VeriMap	92.07 (1.21)^{s}	74.33 (1.63)^{s}	76.93 (0.90)^{s}	54.87 (1.27)^{s}
ARGDes	92.07 (0.83)^{s}	74.47 (0.81)^{s}	77.00 (1.60)^{s}	55.27 (1.92)^{s}
EIB	91.40 (0.72)^{s}	74.60 (1.59)^{s}	76.13 (2.19)^{s}	55.53 (3.16)^{s}

Table 16. Tabular reasoning results with uncertainty terms and source codes.

Method	G-MAS-Complex Strict EM $N = 400$
DeepSeek	$26.75\,(2.21)^{\mathrm{b}}$
Qwen	$13.25\,(1.70)^{\mathrm{b}}$
AutoGen	$23.50\,(2.12)^{\mathrm{b}}$
MADebate	$14.00\,(1.73)^{\mathrm{b}}$
DyLAN	$4.50\,(1.04)^{\mathrm{b}}$
GPTSwarm	$15.50\,(1.81)^{\mathrm{b}}$
GDesigner	$14.50\,(1.76)^{\mathrm{b}}$
SparseCT	$15.25\,(1.80)^{\mathrm{b}}$
GraphSearch	$14.00\,(1.73)^{\mathrm{b}}$
GraphR1	$10.75\,(1.55)^{\mathrm{b}}$
R-GFM	$2.25\,(0.74)^{\mathrm{b}}$
BigMAS	$19.25\,(1.97)^{\mathrm{b}}$
MASRouter	$10.75\,(1.55)^{\mathrm{b}}$
GoAgent	$\mathbf{50.25}\,(2.50)^{\mathrm{b}}$
VeriMap	$\underline{29.50}\,(2.28)^{\mathrm{b}}$
ARGDes	$13.25\,(1.70)^{\mathrm{b}}$
EIB	$14.00\,(1.73)^{\mathrm{b}}$

Table 17. G-MAS-Complex results. Entries give strict exact match in percent with a binomial standard-error estimate in parentheses over 400 tasks.

Sampling variability. The following tables report means and sample standard deviations across three 500-instance sample sets. Their variation is across sampled task sets. These entries reproduce the cells marked s in the detailed results. Bold and underlined means identify the highest and second-highest distinct means in each dataset, including ties.

Method	ASDiv-A	MATH
DeepSeek	88.53 ± 1.10	55.73 ± 3.64
Qwen	86.87 ± 0.42	45.20 ± 2.27
AutoGen	88.53 ± 0.99	56.13 ± 2.66
MADebate	88.53 ± 0.99	55.27 ± 3.42
DyLAN	$\mathbf{88.80} \pm 1.40$	56.07 ± 3.24
GPTSwarm	88.67 ± 0.81	57.00 ± 2.84
GDesigner	88.60 ± 1.31	56.73 ± 3.37
SparseCT	$\mathbf{88.80} \pm 0.92$	$\underline{57.13} \pm 3.87$
GraphSearch	88.67 ± 0.90	56.40 ± 3.46
GraphR1	88.60 ± 1.06	56.73 ± 2.72
R-GFM	$\mathbf{88.80} \pm 1.44$	55.73 ± 3.88
BigMAS	88.53 ± 1.51	$\mathbf{57.47} \pm 3.59$
MASRouter	88.40 ± 1.59	56.00 ± 3.33
GoAgent	88.53 ± 0.99	56.07 ± 2.89
VeriMap	$\underline{88.73} \pm 0.50$	56.33 ± 3.87
ARGDes	88.53 ± 1.17	56.53 ± 3.01
EIB	$\mathbf{88.80} \pm 1.40$	56.60 ± 3.29

Table 18. Mathematical reasoning under sample-set variation. Entries report mean $\pm$ sample standard deviation in percent across three 500-instance sample sets.

Method	MMLU	MMLU-Redux	MMLU-Pro	StrategyQA	Commonsense QA	ARC-Easy
DeepSeek	85.40±0.92	83.73±0.50	73.07±0.76	84.60±0.80	85.40±0.53	94.27±0.61
Qwen	82.33±2.76	79.80±0.92	60.93±0.90	79.40±1.93	85.67±1.33	94.07±0.76
AutoGen	85.80±0.35	84.00±1.73	71.80±1.20	84.13±0.70	85.67±1.30	94.27±0.76
MADebate	85.93±0.12	83.60±0.53	71.87±1.14	84.07±1.96	85.20±1.56	94.07±0.81
DyLAN	86.00±1.39	83.93±1.50	**73.87**±1.21	**86.67**±0.70	85.73±0.61	94.07±0.76
GPTSwarm	85.60±0.53	84.40±0.35	70.73±0.61	85.67±2.84	85.53±0.64	94.27±0.76
GDesigner	85.67±1.51	84.27±1.17	71.87±0.31	85.00±2.03	86.13±1.36	94.20±1.06
SparseCT	85.20±0.87	83.73±0.50	71.33±1.10	85.80±0.80	86.13±1.21	93.93±0.64
GraphSearch	85.80±0.80	84.40±1.91	71.60±0.92	85.60±0.20	86.07±1.27	94.27±0.58
GraphR1	85.33±1.30	84.00±1.06	70.53±1.33	85.60±0.69	85.47±1.67	94.20±0.72
R-GFM	85.47±1.27	84.27±1.62	71.53±0.50	85.13±0.50	85.93±1.30	93.93±0.83
BigMAS	86.00±1.11	84.40±0.92	72.20±0.20	85.07±1.62	85.87±1.03	93.93±1.03
MASRouter	86.53±1.03	84.33±0.83	72.53±1.81	85.67±0.58	86.07±0.70	**94.33**±0.64
GoAgent	85.93±1.14	83.87±1.33	70.33±0.50	85.27±2.05	**86.47**±1.45	94.00±1.06
VeriMap	85.87±0.46	84.13±1.51	73.53±1.70	83.53±1.17	85.40±1.00	93.87±0.92
ARGDes	**87.27**±1.10	83.60±1.04	71.47±0.58	86.00±1.59	86.20±2.31	94.13±0.64
EIB	86.13±1.67	**84.53**±1.42	72.47±0.12	85.40±1.40	85.73±1.17	93.93±0.70

Table 19. Knowledge and commonsense under sample-set variation. Entries report mean ± sample standard deviation in percent across three 500-instance sample sets.

Method	MedMCQA
DeepSeek	79.67±0.99
Qwen	73.53±0.61
AutoGen	79.67±1.70
MADebate	78.53±1.14
DyLAN	79.93±1.10
GPTSwarm	79.00±0.87
GDesigner	79.93±0.76
SparseCT	79.67±1.67
GraphSearch	78.73±1.01
GraphR1	79.13±0.83
R-GFM	79.53±0.95
BigMAS	78.73±1.85
MASRouter	78.80±1.91
GoAgent	79.13±0.92
VeriMap	80.07±1.51
ARGDes	**80.53**±0.70
EIB	78.87±1.33

Table 20. Biomedical reasoning under sample-set variation. Entries report mean ± sample standard deviation in percent across three 500-instance sample sets.

Method	TabFact	WikiSQL	WikiTable Questions	SQA
DeepSeek	92.00±1.11	74.07±0.61	76.73±2.16	55.20±1.44
Qwen	81.40±1.56	74.93±1.27	65.27±1.50	46.53±2.14
AutoGen	91.80±1.22	74.20±1.78	75.93±1.03	54.67±1.97
MADebate	91.47±0.42	74.27±1.10	76.67±1.33	54.60±1.93
DyLAN	**92.53**±1.53	73.87±2.53	77.40±1.44	55.00±1.74
GPTSwarm	92.20±1.40	74.80±1.97	76.60±1.93	56.13±3.13
GDesigner	91.73±0.61	74.93±0.83	76.60±1.06	55.73±4.12
SparseCT	91.93±1.10	74.87±1.22	**77.47**±0.76	56.53±1.86
GraphSearch	92.00±1.56	74.67±0.81	77.13±1.62	55.60±1.44
GraphR1	91.40±1.71	74.40±0.69	76.80±0.20	56.13±1.81
R-GFM	92.13±2.30	**75.60**±1.25	77.40±1.56	56.07±3.67
BigMAS	92.33±1.29	74.73±0.90	76.80±1.11	55.33±3.03
MASRouter	91.40±1.11	74.00±2.55	75.67±0.81	57.60±1.51
GoAgent	92.33±0.81	74.47±1.79	76.20±0.80	**57.73**±2.34
VeriMap	92.07±1.21	74.33±1.63	76.93±0.90	54.87±1.27
ARGDes	92.07±0.83	74.47±0.81	77.00±1.60	55.27±1.92
EIB	91.40±0.72	74.60±1.59	76.13±2.19	55.53±3.16

Table 21. Tabular and structured reasoning under sample-set variation. Entries report mean ± sample standard deviation in percent across three 500-instance sample sets.

A.10 Artifact and Reproducibility

The benchmark artifact separates task loading, baseline construction, backend execution, answer parsing, metric calculation, cost accounting, and intervention diagnosis. Each method is invoked through a common runner, making the raw prediction, parsed answer, metric result, trace, and cost fields comparable across single-agent, ordinary MAS, and graph-enhanced MAS configurations.

Each execution trace stores a method identifier, dataset identifier, sample identifier, organization hash, prompt-template hash, backend configuration, random seed when used, final prediction, parsed prediction, metric result, and cost fields. For every executed unit, the trace records role name, incoming message identifiers, outgoing message identifiers, model-call metadata, token counts, latency, retry status, and termination reason. For intervention runs, the trace additionally stores intervention parameters and changed graph or runtime elements.

The artifact specification includes dataset loaders, evaluation samples, baseline adapters, organization configurations, intervention operators, predictions, traces, cost logs, and aggregation scripts.

Reproducibility records. The artifact contains three linked record types. A configuration record stores the dataset list, sample manifest, backend setting, prompt template, organization hash, and intervention parameters. A scoring record stores the parser version, metric name, metric input, parsed prediction, invalid-parse flag, and score. An aggregation record stores the included datasets, metric weights, uncertainty source, method grouping, and the rule for selecting representative configurations. These records make the full tables auditable and preserve dataset-level metric identities even when results are summarized by domain or method family.

B Supplementary Organizational Diagnosis

B.1 Illustrative Multi-Document Task

This constructed example illustrates the dependency and conflict logic of G-MAS-Complex. The query asks for the available inventory of product P and the identifiers of the records supporting the calculation. The document collection contains the following records.

Record	Content
D1	Initial inventory of product P is 12 units.
D2	Revised inventory of product P is 9 units. This record supersedes D1.
D3	The reservation for product P is 4 units.

The correct answer gives `available_units` as 5. It sets `inventory_source` to D2 and `reservation_source` to D3. Extraction locates the records, revision checking selects D2, calculation obtains $9 - 4 = 5$, and verification checks the value and source identifiers. Two illustrative error cases distinguish these requirements. Selecting D1 produces $12 - 4 = 8$, while returning 5 without D3 omits a required source.

A collaboration graph can assign extraction, revision checking, calculation, and integration to separate units. The illustrative records expose the task's dependency structure and distinguish record selection, numerical calculation, and source reporting within the answer contract.

B.2 Component Scoring and Input Contracts

A diagnostic extension can decompose strict exact match into the following checks. Each component rate uses the full evaluated task list as its denominator. The full-task check retains all constraints of the original answer contract.

Component	Check
Format	The output parses and contains the required fields and types.
Core answer	Requested values and rankings agree with the reference.
Source	Required source identifiers support the returned answer.
Conflict	Selected records follow the task's revision and authority rules.
Full task	All required checks, checkpoints, and checksum constraints hold.

The input contract separates the query, source documents, dependency and conflict information, and answer schema from evaluator annotations. For Γ_i, it identifies relations stated in the input, relations encoded by document contents, and labels available to the evaluator. Each answer field has a type, a reference value, and a validation rule.

The construction record schema contains a task identifier, template identifier, generation procedure, and suite version. Generation fields specify the template or model revision, prompt, sampling settings, and random seed. Tier metadata describes document count, input length, dependency depth, conflicting records, distractors, and required answer fields. Validation fields specify reference and checksum checks, duplicate checks within and across splits, and human review records. Review fields identify the sampled tasks, annotators, decisions, and disagreement resolution. A frozen manifest joins accepted task identifiers with file hashes, the scoring revision, freeze date, and license.

B.3 Reproducibility Record Schema

The following schema defines the information needed to reconstruct a comparison. It specifies fields for dataset manifests, adapter configurations, and execution records. A comparison identifier joins these sources with predictions and scores.

Record group	Required fields
Dataset	Version, split, filters, sample identifiers, order, sampling seed, frozen manifest, and file hashes.
Model	Exact version, request date, reasoning mode, decoding parameters, context limit, and output limit.
Adapter	Implementation revision, configuration hash, retained mechanisms, changes, role prompts, and stopping rules.
Learning	Training and validation manifests, test isolation, objective, checkpoint, and graph-generation procedure.
Tools	Tool permissions, verification access, candidate selection, and format-repair rules.
Shared state	State entries readable and writable by each unit, with message delivery and state access distinguished.
Run and score	Run identifier, random seed, parser and metric versions, score unit, aggregation rule, and reference answers.
Cost and failure	Tokens, calls, retries, latency, timeouts, parsing and tool failures, termination status, concurrency, caching, and hardware.

For candidate replacement, the record includes eligible items, selected items, replacement content, and credibility markers in both reference and modified conditions. For worker failures, it includes the worker population, selection rule, injection times, duration, and whether the count represents simultaneous unavailability or accumulated events. A paired record joins reference and modified executions through the same task and configuration identifiers. It records the intervention type, selected items, random seed, schedule, and achieved modification level. Predictions, scores, and resource records share the execution identifier.

B.4 Organization Analysis Protocol

Graph construction and execution describe different properties. GPTSwarm optimizes prompts and connectivity (Zhuge et al., 2024). G-Designer learns task-dependent graph generation (Zhang et al., 2025). ARG-Designer sequentially generates roles and links (Li et al., 2026). Configuration records can describe these properties through the following attributes.

Attribute	Recorded configuration
Topology	Fixed, generated for each task, or updated during execution.
Learning	Optimized prompts, graph parameters, routing rules, or policies.
Roles	Unit definitions and differences in prompts, tools, and models.
Verification	Checks available to each unit and their invocation rules.
Recovery	Retry, fallback, or rerouting rules and termination conditions.

Graph records associate task identifiers with initial nodes, directed edges, updates, and executed messages. For a simple directed graph with $n \geq 2$ nodes and m edges, density is $m/[n(n-1)]$. Let $\mathcal{R}$ contain reachable ordered pairs of distinct nodes. The reachable-pair fraction and average reachable path length are

$$R_G = \frac{|\mathcal{R}|}{n(n-1)}, \qquad L_G = \frac{1}{|\mathcal{R}|} \sum_{(u,v)\in\mathcal{R}} d_G(u,v). \tag{15}$$

Here d_G is shortest directed path length, and L_G requires $|\mathcal{R}| > 0$. Comparing constructed edges with executed message edges connects available routes with their use. For executions containing messages, let p_v be the fraction originating from node v. Normalized source entropy is

$$H_G = -\frac{1}{\log n} \sum_{v\in V} p_v \log p_v, \tag{16}$$

with zero-probability terms contributing zero. Role-level message counts and temporal records describe communication before and after an intervention.

B.5 Reimplementation Validation Protocol

Reimplementation validation aligns the source revision, task split, backend, prompts, graph construction, optimization, decoding, stopping rule, and metric with the original method. Its compari-

son record contains the published score, reproduced score, their difference, and repetition count. A subsequent benchmark evaluation applies the shared task presentation and scoring controls. Method-specific roles and coordination rules remain part of the adapter description. This sequence connects original-setting validation with evaluation under the benchmark configuration.

B.6 Follow-up Comparisons and Statistical Records

Future budget-matched comparisons can evaluate direct single-agent execution, iterative planning and verification, independent candidate generation, and representative multi-agent configurations under common model, tool, input, and resource limits. Actual consumption accompanies task scores. Candidate selection uses information available to the evaluated system. Role-replacement comparisons can preserve nodes and connections while replacing specialist prompts with general roles or exchanging role prompts. These settings test role definitions.

Further rewiring comparisons can check output reachability, scheduling validity, and schema compatibility for each proposed organization, alongside degree preservation. Repeated graph samples record seeds, accepted swaps, actual replacement ratios, and execution costs. Recovery comparisons can apply an identical failure schedule to the same configuration with recovery enabled and disabled, recording accuracy, retries, tokens, and latency.

Paired statistical analysis resamples common task identifiers and computes score differences within each replicate. The resampling specification groups generated tasks by template family to account for dependence among related instances. The analysis record specifies the resampling unit, repetition count, interval method, and central estimate. Separate run identifiers track execution randomness, while sample manifests track variation from dataset selection.

References

Hongjiang Chen, Xin Zheng, Yixin Liu, Pengfei Jiao, Shiyuan Li, Huan Liu, Zhidong Zhao, Ziqi Xu, Ibrahim Khalil, and Shirui Pan. Goagent: Group-of-agents communication topology generation for llm-based multi-agent systems, 2026. URL https://arxiv.org/abs/2603.19677.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021a. URL https://arxiv.org/abs/2107.03374.

Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and William Yang Wang. Tabfact: A large-scale dataset for table-based fact verification, 2020. URL https://arxiv.org/abs/1909.02164.

Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. FinQA: A dataset of numerical reasoning over financial data. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 3697–3711, Online and Punta Cana, Dominican Republic, November 2021b. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.300. URL https://aclanthology.org/2021.emnlp-main.300/.

Zhiyu Chen, Shiyang Li, Charese Smiley, Zhiqiang Ma, Sameena Shah, and William Yang Wang. ConvFinQA: Exploring the chain of numerical reasoning in conversational finance question answering. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 6279–6292, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.421. URL https://aclanthology.org/2022.emnlp-main.421/.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge, 2018. URL https://arxiv.org/abs/1803.05457.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *CoRR*, abs/2110.14168, 2021. URL https://arxiv.org/abs/2110.14168.

DeepSeek-AI. Deepseek-v3 technical report, 2025. URL https://arxiv.org/abs/2412.19437.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023. URL https://arxiv.org/abs/2305.14325.

Aryo Pradipta Gema, Joshua Ong Jun Leang, Giwon Hong, Alessio Devoto, Alberto Carlo Maria Mancino, Rohit Saxena, Xuanli He, Yu Zhao, Xiaotang Du, Mohammad Reza Ghasemi Madani, Claire Barale, Robert McHardy, Joshua Harris, Jean Kaddour, Emile van Krieken, and Pasquale Minervini. Are we done with mmlu?, 2025. URL https://arxiv.org/abs/2406.04127.

Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics*, 9:346–361, 2021. doi: 10.1162/tacl_a_00370. URL https://aclanthology.org/2021.tacl-1.21/.

Guangfu Hao, Yuming Dai, Xianzhe Qin, and Shan Yu. Brain-inspired graph multi-agent systems for llm reasoning, 2026. URL https://arxiv.org/abs/2603.15371.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2021a. URL https://arxiv.org/abs/2009.03300.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *CoRR*, abs/2103.03874, 2021b. URL https://arxiv.org/abs/2103.03874.

Mohit Iyyer, Wen-tau Yih, and Ming-Wei Chang. Search-based neural structures for sequential question answering. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pp. 209–220. Association for Computational Linguistics, 2017. doi: 10.18653/v1/P17-1020. URL https://aclanthology.org/P17-1020/.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL https://arxiv.org/abs/2403.07974.

Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 11(14):6421, 2021. doi: 10.3390/app11146421. URL https://www.mdpi.com/2076-3417/11/14/6421.

Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. PubMedQA: A dataset for biomedical research question answering. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (eds.), *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 2567–2577, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1259. URL https://aclanthology.org/D19-1259/.

Shiyuan Li, Yixin Liu, Qingsong Wen, Chengqi Zhang, and Shirui Pan. Assemble your crew: Automatic multi-agent communication topology design via autoregressive graph generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 23142–23150, 2026.

Yunxuan Li, Yibing Du, Jiageng Zhang, Le Hou, Peter Grabowski, Yeqing Li, and Eugene Ie. Improving multi-agent debate with sparse communication topology. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 7281–7294, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.427. URL https://aclanthology.org/2024.findings-emnlp.427/.

Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. Program induction by rationale generation: Learning to solve and explain algebraic word problems. In Regina Barzilay and Min-Yen Kan (eds.), *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 158–167, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1015. URL https://aclanthology.org/P17-1015/.

Haokun Liu, Zezhong Ding, and Xike Xie. Learning graph foundation models on riemannian graph-of-graphs, 2026a. URL https://arxiv.org/abs/2605.09993.

Jiajin Liu, Yuanfu Sun, Dongzhe Fan, and Qiaoyu Tan. Graphsearch: Agentic search-augmented reasoning for zero-shot graph learning, 2026b. URL https://arxiv.org/abs/2601.08621.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, 2023. URL https://arxiv.org/abs/2305.01210.

Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. A dynamic llm-powered agent network for task-oriented agent collaboration, 2024. URL https://arxiv.org/abs/2310.02170.

Haoran Luo, Haihong E, Guanting Chen, Qika Lin, Yikai Guo, Fangzhi Xu, Zemin Kuang, Meina Song, Xiaobao Wu, Yifan Zhu, and Luu Anh Tuan. Graph-r1: Towards agentic graphrag framework via end-to-end reinforcement learning, 2026. URL https://arxiv.org/abs/2507.21892.

Shen-yun Miao, Chao-Chun Liang, and Keh-Yih Su. A diverse corpus for evaluating and developing English math word problem solvers. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 975–984, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.92. URL https://aclanthology.org/2020.acl-main.92/.

Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In Gerardo Flores, George H Chen, Tom Pollard, Joyce C Ho, and Tristan Naumann (eds.), *Proceedings of the Conference on Health, Inference, and Learning*, volume 174 of *Proceedings of Machine Learning Research*, pp. 248–260. PMLR, 07–08 Apr 2022. URL https://proceedings.mlr.press/v174/pal22a.html.

Panupong Pasupat and Percy Liang. Compositional semantic parsing on semi-structured tables. In Chengqing Zong and Michael Strube (eds.), *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 1470–1480, Beijing, China, July 2015. Association for Computational Linguistics. doi: 10.3115/v1/P15-1142. URL https://aclanthology.org/P15-1142/.

Arkil Patel, Satwik Bhattamishra, and Navin Goyal. Are NLP models really able to solve simple math word problems? In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (eds.), *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2080–2094, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.168. URL https://aclanthology.org/2021.naacl-main.168/.

Qwen Team. Qwen2.5 technical report, 2025. URL https://arxiv.org/abs/2412.15115.

Subhro Roy and Dan Roth. Solving general arithmetic word problems. In Lluís Màrquez, Chris Callison-Burch, and Jian Su (eds.), *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pp. 1743–1752, Lisbon, Portugal, September 2015. Association for Computational Linguistics. doi: 10.18653/v1/D15-1202. URL https://aclanthology.org/D15-1202/.

Xu Shen, Yixin Liu, Yiwei Dai, Yili Wang, Rui Miao, Yue Tan, Shirui Pan, and Xin Wang. Understanding the information propagation effects of communication topologies in llm-based multi-agent systems, 2025. URL https://arxiv.org/abs/2505.23352.

Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In Jill Burstein, Christy Doran, and Thamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4149–4158, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1421. URL https://aclanthology.org/N19-1421/.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark, 2024. URL https://arxiv.org/abs/2406.01574.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL https://arxiv.org/abs/2308.08155.

Tianyang Xu, Dan Zhang, Kushan Mitra, and Estevam Hruschka. Verification-aware planning for multi-agent systems, 2025. URL https://arxiv.org/abs/2510.17109.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023. URL https://arxiv.org/abs/2305.10601.

Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, Kun Wang, Dawei Cheng, and Yiyan Qi. Masrouter: Learning to route llms for multi-agent systems, 2025. URL https://arxiv.org/abs/2502.11133.

Guibin Zhang, Yanwei Yue, Xiangguo Sun, Guancheng Wan, Miao Yu, Junfeng Fang, Kun Wang, Tianlong Chen, and Dawei Cheng. G-designer: Architecting multi-agent communication topologies via graph neural networks, 2025. URL https://arxiv.org/abs/2410.11782.

Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning, 2017. URL https://arxiv.org/abs/1709.00103.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (eds.), *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 3277–3287, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.254. URL https://aclanthology.org/2021.acl-long.254/.

Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. MultiAgentBench : Evaluating the collaboration and competition of LLM agents. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8580–8622, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.421. URL https://aclanthology.org/2025.acl-long.421/.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Language agents as optimizable graphs, 2024. URL https://arxiv.org/abs/2402.16823.